When Can the Planner Trust a Filtered Unique Index?

Ziheng Wei
School of Computer Science, Wuhan University
Wuhan, Hubei, China
ziheng.wei@whu.edu.cn
ORCID: 0000-0002-1859-0374

Sebastian Link
School of Computer Science, The University of Auckland
Auckland, New Zealand
s.link@auckland.ac.nz
ORCID: 0000-0002-1816-2863

***Abstract*—Applications declare keys that hold on part of a table—among undeleted rows, current versions, primary addresses—as filtered unique indexes: one in nine unique indexes in ten open-source application schemas. Optimisers do not trust them: PostgreSQL did, returned wrong results, and since 2023 ignores them, as do CockroachDB and SQL Server; SQLite matches predicates syntactically. We show when trust is sound. A proof may use only the table's own restrictions, and it must use the `null` semantics of the rewrite: join-shaped rewrites need uniqueness with `null`s distinct, group-shaped ones with `null`s not distinct. The first question is polynomial for conjunctive predicates without inequalities or `IN`-lists on the columns that both compared rows must fill; the second is coNP-complete, and under the same conditions fixed-parameter tractable in the open nullable key columns. Among the indexes of real schemas, 2,953 of 2,960 implications are as easy as the first. Our prover combines all unique indexes and `CHECK` constraints of a table, carries uniqueness through joins, and fits into PostgreSQL 18.6 through 38 planner lines, adding 0.03 ms, and into CockroachDB. On 1,000,000 random queries it changed 154,532 plans and no result; on catalogs over 226 real tables it licenses exactly the rewrites they imply, where PostgreSQL licenses none and SQLite a sixth of the joins. It speeds up users' queries over Stack Exchange by up to $3.3\times$, mostly through keys carried through joins, and queries we wrote over MusicBrainz by up to $4.9\times$, where a filtered index states a key that PostgreSQL ignores.**

***Index Terms*—query optimisation, filtered unique indexes, partial indexes, keys, null markers, implication**

I. Introduction

Applications rarely need a key to hold on a whole table. A user's e-mail address must be unique among accounts that are not deleted; an item identifier among current versions of a slowly changing dimension; a template name among the templates of one account, of one inbox, or of the installation. SQL's `UNIQUE` constraint cannot say this. Database systems offer filtered unique indexes instead:

```
CREATE UNIQUE INDEX ON users (email) WHERE
        deleted_at IS NULL;
```

PostgreSQL calls them partial indexes [50], [41], SQL Server filtered indexes [46], SQLite partial indexes [45]; Rails and Django generate them from model declarations. In ten open-source application schemas we found 174 of them among 1,513 unique indexes, in seven of the ten applications (Section II).

A unique index is also a fact the optimiser can use. If the inner side of a left join matches each outer row at most once and no column of it is used, the join can be dropped; if the subquery of an `IN` predicate returns each value at most once, the semi-join is a join; if the grouping columns of a block contain a key, `GROUP BY` is redundant [44], [47]. Each of the four systems applies some of these rewrites for unconditional unique indexes. For filtered ones the picture is different (Section II-B):

- *PostgreSQL* used a partial unique index as a uniqueness proof whenever the query's clauses implied its predicate. Because those clauses included join conditions, it returned wrong results (bug #17975), and since June 2023 every supported version ignores partial indexes in all of its uniqueness-based optimisations [42]—even when the query predicate *is* the index predicate. Users have asked for these rewrites: a 2019 report shows a left join kept over `UNIQUE (sub_id) WHERE deleted_at IS NULL` [14], and a 2026 patch proposal proves partial-index predicates from base restrictions only, for estimates, and leaves uniqueness proofs open [15].
- *SQLite* removes a left join over a partial unique index if, for every conjunct of the index predicate, one term of the query matches it syntactically; `st = 0` does not match `st IN (0, 1)`. It never uses partial indexes to remove `DISTINCT`.
- *CockroachDB* leaves partial unique indexes out of a table's keys and removes no join, `DISTINCT` or grouping column over them, even directly above a scan of the index.
- *SQL Server* likewise removes no join, `DISTINCT` or grouping over a filtered unique index, and costs joins over one as many-to-many [52].

The systems are not wrong to be careful. There are two pitfalls, and each has bitten. The first is PostgreSQL's: a uniqueness proof must hold *before* joins, so it may use only restrictions of the base table. The second concerns `null`: a filtered unique index in PostgreSQL or SQLite allows any number of rows with a `null` key, so it proves that a join probe finds at most one partner but not that `DISTINCT` is redundant—`DISTINCT` puts all `null`s into one group. Both pitfalls are

users

id	email	deleted_at
1	a@x	null
2	a@x	2024-03-01
3	b@x	null

user_emails

user_id	address	is_primary
1	a@x	true
1	a2@y	false
3	b@x	true

```
UNIQUE (email) WHERE deleted_at IS NULL
  {⟨deleted_at: ⊥⟩} : U(email), email NOT NULL
UNIQUE (user_id, is_primary) WHERE is_primary
  {⟨is_primary: true⟩} : U(user_id, is_primary)

SELECT DISTINCT u.email, e.address FROM users u
  JOIN user_emails e ON e.user_id = u.id
    AND e.is_primary
  WHERE u.deleted_at IS NULL
```

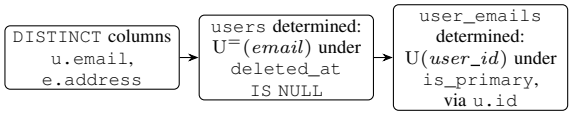


Fig. 1. Running example. Each live user has one e-mail address and each user one primary address, so every row of the join is determined by its DISTINCT values and DISTINCT is redundant. Without e.is_primary, user 1 has two rows; without deleted_at IS NULL, users 1 and 2 share a@x. If email were nullable, U(*email*) would not give U$^{=}$(*email*), as DISTINCT puts all nulls into one group.

real. On random schemas, proving an index predicate from another join's clause, as PostgreSQL did, changes 442 of 20,000 results; on 217 open-data tables, ignoring the null semantics would remove 22,171 DISTINCTs, 3,121 of them wrongly, on 133 of the tables (Section VII).

Approach: We read a filtered unique index as a constraint, a *conditional UNIQUE*, and the planner's question as implication (Fig. 1): do the filtered unique indexes of a table, with its NOT NULL columns and CHECK constraints, imply that the rows matching the query's restriction of the table are unique on the columns the query probes or groups by? Asking it with the restrictions of the base table only avoids the first pitfall. Asking it with the null semantics that the rewrite needs avoids the second. The two semantics also separate the complexity: join-shaped rewrites are polynomial for conjunctive predicates without inequalities or IN-lists on the columns that both compared rows must fill, group-shaped ones are coNP-complete, and among the indexes of real schemas 2,953 of 2,960 implications are as easy as the first case.

Results: The prover is 1,683 lines of C, called from PostgreSQL 18.6's planner through 38 changed lines. *It is sound:* 1,000,000 random queries changed 154,532 plans and no result, and dropping any safeguard is caught by random queries or a feature test. *It is exact on real catalogs:* on 1,454 left joins and 1,454 DISTINCTs over real data with up to 166 filtered indexes per table, it licenses exactly the 1,094 and 171 rewrites the catalog implies, 61 and 54 more than one index at a time; on 217 open-data tables, 1,483 and 963 more; a port to CockroachDB comes within 2 of the 1,231 rewrites its catalog implies; on real workloads one index at a time would have sufficed, and the theory is what makes that check safe. *It is cheap:* 0.03 ms of planning in the median. *It is worth turning on:* beyond the rewrites PostgreSQL has, it removes DISTINCT and reduces GROUP BY through joins, which PostgreSQL never does. Its gains depend on who writes the queries: application code, dbt projects and reports rarely filter on the scopes; on users' Stack Exchange queries it changes 118 plans and gains up to $3.3\times$, mostly through keys carried through joins, as filtered keys gain there only through the Inner Unique setting that we leave off; it removes every join that views over versioned data add; and on queries we wrote over MusicBrainz it gains up to $4.9\times$.

Contributions:

1) *The two pitfalls, made precise.* A study of filtered unique indexes in ten application schemas and of what PostgreSQL 18, SQLite, CockroachDB and SQL Server do with them (Section II), and the two conditions a sound proof needs: base restrictions only, and the rewrite's null semantics.
2) *The complexity of the planner's question.* Conditional UNIQUE constraints under both null semantics of SQL: implication is polynomial for join-shaped rewrites with conjunctive indexes under two side conditions, coNP-complete for group-shaped ones and, under the same conditions, fixed-parameter tractable, and coNP-complete when either condition fails on a key column (Sections III and IV). A finite axiomatisation and two-record counterexamples for failed implications are in the appendix.
3) *A sound prover* in PostgreSQL 18.6, exact for implication on one relation over the indexes it translates, up to a search limit, a template limit and a premise filter, serving its join rewrites and GROUP BY reduction, adding DISTINCT removal, reading CHECK constraints, and carrying both semantics through inner, left, semi- and anti-joins, and a port to CockroachDB that adds the keys it proves to the optimiser's functional dependencies (Section V).
4) *Evidence* of soundness, of exactness against four systems on real catalogs, of cost, and of gains on workloads by who writes them, from application code to users' queries over Stack Exchange, SkyServer and Wikimedia (Section VII).

The appendix contains the proofs, the axiomatisation, a recommender for filtered unique indexes and further experiments.

II. Filtered Unique Indexes in Practice

A. In application schemas

We took the schema dumps of ten open-source applications that use PostgreSQL—GitLab (structure.sql), Mastodon, Forem, Chatwoot, Loomio, Consul, Open Food Network, Maybe, DocuSeal and Campsite (Rails schema.rb)—and parsed their unique indexes, primary keys, UNIQUE constraints and NOT NULL columns (GitLab's per-partition copies of an index are counted once). Table I summarises them.

The predicates fall into few shapes:

TABLE I
UNIQUE INDEXES IN TEN APPLICATION SCHEMAS

Application	Tables	Unique	Filtered	≥ 2 per table
GitLab	1,135	930	128	26
Mastodon	122	91	14	3
Forem	131	110	12	4
Chatwoot	109	93	9	2
Loomio	76	68	5	1
DocuSeal	44	36	3	0
Maybe	59	30	3	1
Three others*	323	155	0	0
Total	1,999	1,513	174	37

*Campsite, Consul, Open Food Network

- *Excluding `null` keys* (87): `UNIQUE (k) WHERE k IS NOT NULL`; all of Mastodon's 14 and 63 of GitLab's. Under `NULLS DISTINCT` the predicate changes nothing—such an index states an unconditional key—and serves only to keep the index small, or is the habit of SQL Server, where it is the only way to allow several `null`s.
- *A constant or flag* (33): `status = 0`, `published IS TRUE`, `rule_type = 2`.
- *A missing value on another column* (26): `ended_at IS NULL`, `resolved_at IS NULL`—soft deletion, current versions, open episodes.
- *IN-lists* (9): `status = ANY (ARRAY[0, 1])`.
- *Conjunctions of the above* (5), e.g. Chatwoot's three scopes for e-mail templates: per installation (`account_id IS NULL AND inbox_id IS NULL`), per account (`account_id IS NOT NULL AND inbox_id IS NULL`) and per inbox.
- *Inequalities* (14): `package_type = 7 AND status <> 4`, `custom_domain <> ''`.

Six indexes have expression keys such as `lower(email)`; all 168 others have predicates built from `=`, `<>`, `IN`, `IS [NOT] NULL`, `AND` and `OR`, and all 168 are one-template conditions in the sense of Section III: 159 conjunctions and 9 `IN`-lists, which SQL writes as disjunctions but a template holds in one entry. 37 tables carry two or more filtered unique indexes, where reasoning about one at a time can miss what they say together. Only two filtered indexes are implied by others (both by the primary key); filtered indexes are not declared redundantly. Beyond these applications, in the SQL files of SchemaPile [7] (its 22,989 permissively licensed schemas from GitHub) filtered indexes are 12.9% of the `CREATE UNIQUE INDEX` statements in SQL Server files (118 of 913) and 5.4% in PostgreSQL files (182 of 3,362). Their predicates have the same shapes: constants and flags, `null` tests on other columns, and the exclusion of `null` keys, with conjunctions far more common than `OR` (69 against 8 of the 356 filtered unique indexes in all dialects) and 6 range comparisons.

In analytics code and open data: Warehouses built with dbt assert keys as tests, and in the two largest public dbt projects we found, 38 of 369 uniqueness tests carry a scope (`where:`), 27 of them keys with documented exceptions.

In open data: MusicBrainz publishes its PostgreSQL schema and data, with 17 filtered unique indexes: at most one primary alias per entity and locale, `(entity, locale) WHERE primary_for_locale AND locale IS NOT NULL`, on 13 alias tables, and artists and labels unique by name split by a comment, `(name) WHERE comment IS NULL` and `(name, comment) WHERE comment IS NOT NULL`, the encoding of $\mathrm{U}^{=}()$ of Proposition 3. Since `comment` became `NOT NULL`, the second states an unconditional key, which PostgreSQL ignores for being partial. Every key holds in the data of 28 October 2026.

TABLE II
UNIQUENESS-BASED OPTIMISATIONS WITH A PLAIN AND A FILTERED UNIQUE INDEX WHOSE PREDICATE EQUALS THE QUERY PREDICATE

System	Optimisation	plain	filtered
PostgreSQL 18	left-join removal	yes	no
	semi-join to join	yes	no
	inner-unique join	yes	no
	`GROUP BY` column removal	yes	no
	self-join elimination	yes	no
	`DISTINCT` removal	no	no
SQLite 3.54	`DISTINCT` removal	yes	no
	left-join omission	yes	syntactic
CockroachDB	left-join removal	yes	no
	`DISTINCT` removal	yes	no
	`GROUP BY` reduction	yes	no
SQL Server	left-join removal	yes	no
	semi-join to join	yes	no
	`DISTINCT` removal	yes	no
	`GROUP BY` removal	yes	no

B. In optimisers

Table II lists the uniqueness-based optimisations of PostgreSQL 18.6, SQLite 3.54, CockroachDB 26.3 and SQL Server 2025 and whether a filtered unique index enables them, measured with an unconditional index on k and a partial index on k `WHERE d IS NULL`, queried with exactly that predicate, and traced to the source.

PostgreSQL: Before planning, `check_index_predicates` sets a partial index's `predOK` flag if `predicate_implied_by` proves its predicate from the relation's restrictions. Until 2023, `relation_has_unique_index_for`—the uniqueness test behind left-join removal, semi-join reduction and inner-unique joins—accepted partial indexes with `predOK` set. But `predOK` may be proved from join clauses that are evaluated only after the inner relation is read, so the planner could treat a relation as unique that is not. The fix (commit `7fcd7ef2a9`, back-patched to all supported versions) excludes every index with a predicate from `relation_has_unique_index_for` and `rel_supports_distinctness`; the reduction of `GROUP BY` by unique indexes and self-join elimination, both

added in version 18, skip partial indexes from the start. Partial indexes with `predOK` remain only in the estimate of the number of distinct values. PostgreSQL removes `DISTINCT` for no base table, filtered or not.

SQLite: `isDistinctRedundant` skips partial indexes. Left-join omission uses the one-row flag of the access path chosen for the inner table, which a partial unique index gets when `whereUsablePartialIndex` finds, for each conjunct of the predicate, one term of the `ON` or `WHERE` clause on that table that implies it by `sqlite3ExprImpliesExpr`: the same expression, one branch of an `OR`, or `IS NOT NULL` after a null-rejecting comparison. This is sound but fragile: `b.st = 0` against `WHERE st IN (0, 1)` and `NOT (b.d IS NOT NULL)` against `WHERE d IS NULL` keep the join, and two indexes are never combined.

CockroachDB: The optimiser derives a table's keys in `MakeTableFuncDep`, which skips partial unique indexes and partial `UNIQUE WITHOUT INDEX` constraints, as their keys hold on part of the table only. It adds such a key to a scan *of* the partial index, but that scan is an alternative found during exploration, while join elimination, `DISTINCT` removal and grouping reduction are normalisation rules applied to the table's canonical scan. None of them used a partial key in our probes: `DISTINCT k` stays above a scan of the partial unique index on `k`, also for `UNIQUE (k) WHERE k IS NOT NULL`.

SQL Server: Filtered unique indexes have conjunctive predicates of comparisons and `IN`-lists (range comparisons among them are outside our language, so an index with one would not serve as a premise), and `UNIQUE` treats `null` as a value [46]. White [52] shows that the optimiser costs a one-to-one join over filtered unique indexes as many-to-many, and recommends redundant predicates or indexed views as workarounds. We probed SQL Server 2025 (RTM-CU10, Developer edition) as the other systems. With a unique constraint it removes the left join and `DISTINCT`, reduces `GROUP BY` and turns the semi-join into a join; with a filtered unique index on the query's own predicate it does none of these, and it keeps the aggregate for `DISTINCT k` even when it reads the filtered index `UNIQUE (k) WHERE k IS NOT NULL`.

III. Conditional UNIQUE

A table schema is a finite set R of columns with a subset $NN \subseteq R$ declared `NOT NULL`. A *table* over R is a finite multiset of records, which map each column to a value or to `null`, and `null` on no column of NN; duplicates are allowed, as in SQL.

Predicates as templates: A *template* τ maps each column to an entry: $\square$ (anything), $\top$ (`IS NOT NULL`), $\bot$ (`IS NULL`), $\in S$ (`= c` or `IN (S)`) or $\notin S$ (`<> c` or `NOT IN (S)`), for a finite non-empty set S of constants. A `null` matches only $\square$ and $\bot$—a comparison with `null` is never true in a `WHERE` clause—and a value v matches $\square$, $\top$, $\in S$ if $v \in S$, and $\notin S$ if $v \notin S$. A record *matches* τ if it matches every entry. A *condition* is a finite set of templates, matched by a record that matches one of them. Every predicate built from `=`, `<>`, `IN`, `NOT IN`, `IS [NOT] NULL`, `AND` and `OR` over columns and constants is a condition (a disjunctive normal form; conjunctions of entries on one column are again entries or contradictory), and a conjunctive predicate is a condition with one template. Range comparisons are outside the language. We write $\langle d\colon \bot\rangle$ for the template with $\bot$ on d and $\square$ elsewhere.

Definition 1. A *conditional* `UNIQUE` *constraint* has the form $C : \mathrm{U}(K)$ or $C : \mathrm{U}^{=}(K)$ for a condition C and a set $K \subseteq R$. A table r satisfies it if no two records of r (at different positions) both match C and agree on K, where for $\mathrm{U}(K)$ (`NULLS DISTINCT`) agreeing means having the same value on every column of K, and for $\mathrm{U}^{=}(K)$ (`NULLS NOT DISTINCT`) also counts `null` as equal to `null`.

The filtered unique index of the introduction is $\{\langle \mathit{deleted_at}\colon \bot\rangle\} : \mathrm{U}(\mathit{email})$. PostgreSQL's and SQLite's `CREATE UNIQUE INDEX ... WHERE P` is $C_P : \mathrm{U}(K)$, PostgreSQL's `NULLS NOT DISTINCT` variant and SQL Server's filtered unique index are $C_P : \mathrm{U}^{=}(K)$, and a primary key is $\{\langle\rangle\} : \mathrm{U}(K)$ with $K \subseteq NN$. A key over no columns, $C : \mathrm{U}(\emptyset)$, says that at most one record matches C; PostgreSQL writes it as a unique index on a constant expression.

Lemma 2 (Pairs)**.** *A set Σ of conditional `UNIQUE` constraints implies φ iff every table of at most two records that satisfies Σ satisfies φ.*

Proof. Violations are witnessed by two records, and every constraint holds on a sub-table of a table that satisfies it. $\square$

Between the dialects: The two `null` semantics are inter-definable, which is what a migration between systems needs. For a nullable key column, $\mathrm{U}()$ is $\mathrm{U}^{=}()$ restricted to records without `null` on K—the idiom `WHERE k IS NOT NULL` that SQL Server users write. Conversely, SQLite and PostgreSQL before version 15 can express $\mathrm{U}^{=}()$ only by one index per pattern of `null`s on the key:

Proposition 3 (Translation)**.** *Let F be the nullable columns of K. Then*

$$C : \mathrm{U}(K) \equiv C \wedge \langle F\colon \top\rangle : \mathrm{U}^{=}(K),$$
$$C : \mathrm{U}^{=}(K) \equiv \{\, C \wedge p_N : \mathrm{U}(K \setminus N) \;:\; N \subseteq F \,\},$$

where p_N has $\bot$ on N and $\top$ on $F \setminus N$, and $C \wedge p$ refines every template of C by p (dropping contradictory ones). The second equivalence uses $2^{|F|}$ constraints, one per pattern of `null`s on the nullable key columns.

Proof. By Lemma 2 it suffices to compare which pairs of records the two sides forbid. First equivalence: a pair that matches C and agrees on K with values has no `null` on K, so it matches $C \wedge \langle F\colon \top\rangle$ and agrees on K with `null` equal to `null`; conversely, a pair that matches $C \wedge \langle F\colon \top\rangle$ has no `null` on F, and none on $K \setminus F \subseteq NN$, so agreeing on K with `null` equal to `null` is agreeing with values. Second

equivalence: if two records match C and agree on K with `null` equal to `null`, then on each column of K both are `null` or both carry the same value, so they have the same set $N \subseteq F$ of `null` columns on K, both match $C \wedge p_N$, and they agree on $K \setminus N$ with values; they violate the translated constraint for N. Conversely, two records that match $C \wedge p_N$ and agree on $K \setminus N$ with values are `null` on N in both, so they match C and agree on K with `null` equal to `null`. □

Whether fewer than $2^{|F|}$ nulls-distinct constraints can express $C : \mathrm{U}^{=}(K)$ in general is open; we checked both equivalences on 5,000 random instances.

Conditional `UNIQUE` strictly generalises the embedded uniqueness constraints of [54], which are the case $\{\langle E{:}\top\rangle\} : \mathrm{U}(U)$ with $U \subseteq E$: no set of those is equivalent to $\{\langle d{:}\bot\rangle\} : \mathrm{U}(k)$ or to $\{\langle s{:}\in\{c\}\rangle\} : \mathrm{U}(k)$. It is incomparable with conditional functional dependencies [28]: two identical records satisfy every CFD and violate every conditional `UNIQUE` whose scope contains them, and a one-record table satisfies every conditional `UNIQUE` but not every CFD (Appendix A).

IV. Reasoning

Every rewrite of Section V asks one question: given the filtered unique indexes Σ of a table, its `NOT NULL` columns, and the restriction C that a query block places on the table, does every table satisfying Σ have at most one record per value of the columns K among those matching C? Which equality is meant depends on the rewrite:

- *Join-shaped* rewrites—left-join removal, semi-join to join, inner-unique joins, self-join elimination—compare K with an equality predicate, which never matches `null`. They need $\Sigma \models C : \mathrm{U}(K)$.
- *Group-shaped* rewrites—removing `DISTINCT`, removing `GROUP BY` or grouping columns—put all `null`s into one group. They need $\Sigma \models C : \mathrm{U}^{=}(K)$.

Naive use of a filtered index answers the first question for the second, which is the `null` pitfall of the introduction. The complexity, too, is set by the rewrite.

Example 4. Premises of both semantics combine. Let k be nullable. $\{\langle\rangle\} : \mathrm{U}(k)$ alone does not imply $\{\langle\rangle\} : \mathrm{U}^{=}(k)$, as it admits two rows with k `null`, so a `DISTINCT k` cannot be removed. Together with $\{\langle k{:}\bot\rangle\} : \mathrm{U}(\emptyset)$—at most one row with k `null`, in PostgreSQL a unique index on a constant `WHERE k IS NULL`, an expression key our prover does not read—it does: two rows that agree on k with `null` equal to `null` either share a value, which the first index forbids, or are both `null`, which the second forbids. Conversely, scopes do not combine by union: $\{\langle d{:}\bot\rangle\} : \mathrm{U}^{=}(k)$ and $\{\langle d{:}\top\rangle\} : \mathrm{U}(k)$ cover every row, yet imply neither $\{\langle x{:}\in\{1\}\rangle\} : \mathrm{U}(k)$ nor $\{\langle x{:}\in\{1\}\rangle\} : \mathrm{U}^{=}(k)$: two rows with $x = 1$ and the same k, one with d `null` and one without, lie in no common scope.

Call a column *value-forced* for templates τ_i, τ_j of the goal if a pair violating $C : \mathrm{U}(K)$ with t_1 matching τ_i and t_2 matching τ_j must carry a value in both records on it: a column of K or of NN, or one on which both templates have $\top$, $\in S$ or $\notin S$.

Theorem 5 (Join-shaped rewrites)**.** *Let every constraint of Σ have a one-template condition (any mix of $\mathrm{U}()$ and $\mathrm{U}^{=}()$), and let C be a condition such that, for all $\tau_i, \tau_j \in C$, no constraint of Σ has a $\notin$-entry on a value-forced column and neither template has an entry $\in S$ with $|S| \geq 2$ on such a column. Whether $\Sigma \models C : \mathrm{U}(K)$ is decidable in time $O(|C|^2 \cdot |\Sigma| \cdot |R|)$.*

Proof. By Lemma 2, $\Sigma \not\models C : \mathrm{U}(K)$ iff some pair t_1, t_2 with t_1 matching $\tau_i \in C$ and t_2 matching $\tau_j \in C$, equal and `null`-free on K, satisfies Σ. A pair satisfies a one-template constraint $\sigma = \{\rho\} : \mathrm{U}(K_\sigma)$ iff some column A *kills* σ: $t_1[A]$ or $t_2[A]$ fails $\rho(A)$, or $A \in K_\sigma$ and $t_1[A], t_2[A]$ are not equal values (for $\mathrm{U}^{=}()$: not equal values and not both `null`). Killing is thus a disjunction over columns, and what A kills depends only on $(t_1[A], t_2[A])$. Call $(t_1[A], t_2[A])$ *admissible* if it matches $\tau_i(A)$ and $\tau_j(A)$, respects NN, and, for $A \in K$, consists of two equal values. We claim that for every column one admissible state kills everything any admissible state kills; the pair made of these states then satisfies Σ iff some admissible pair does. Let a *fresh* value be one that no constraint mentions. For $A \notin K$: if one record may be `null` and the other a value, a `null` and a fresh value (or the constant the template forces) fail every entry other than □—the `null` fails $\top$, $\in S$ and $\notin S$, the value fails $\bot$ and $\in S$—and separate the records for both semantics. Otherwise the column is value-forced or forced to `null`. If value-forced, two different values, fresh unless the templates fix them to single constants, fail $\bot$ and every $\in S$ that a fresh value fails, and separate the records; no constraint has a $\notin$-entry there, so nothing else can be killed. For $A \in K$ the records share a value, fresh unless a template fixes it to a single constant, which again kills everything a value can kill there. Building and checking the $|C|^2$ pairs takes the stated time. □

Theorem 6 (Group-shaped rewrites)**.** *Deciding $\Sigma \models C : \mathrm{U}^{=}(K)$ is coNP-complete, already when C and all constraints of Σ have one template each. Under the side conditions of Theorem 5 it is decidable in time $O(2^b \cdot |C|^2 \cdot |\Sigma| \cdot |R|)$, where b is the number of columns of K that are neither `NOT NULL` nor fixed to $\top$, $\bot$ or a constant by C.*

Proof. Membership: guess a pair (Lemma 2). For the bound, a pair violating $C : \mathrm{U}^{=}(K)$ must, on each of the b open key columns, be `null` in both records or share a value; the two states kill incomparable sets ($\top$, constants and $\mathrm{U}()$-keys on that column, against $\bot$), and every other column is as in Theorem 5, so 2^b canonical pairs per pair of templates suffice. Hardness: from CNF satisfiability over variables $x_1, \ldots, x_n$, take columns $A_1, \ldots, A_n$ (nullable) and $B \in NN$, the goal $\{\langle\rangle\} : \mathrm{U}^{=}(A_1 \cdots A_n B)$, and for each clause one constraint $\{\rho\} : \mathrm{U}(B)$ with $\rho(A_i) = \bot$ if x_i occurs positively, $\top$ if negatively, and □ otherwise. A pair violating the goal agrees on all A_i, `null` ("x_i false") or a shared value ("true"), and satisfies the clause constraint iff some literal kills it, i.e. iff the assignment satisfies the clause. Hence $\Sigma \models \varphi$ iff the CNF is unsatisfiable. The constraints are equivalent to $\mathrm{U}^{=}(B)$ as $B \in NN$, so hardness holds in SQL Server's semantics alone,

and they meet the side conditions of Theorem 5. Without these, b does not bound the search: Theorem 7 below is hard with $K \subseteq NN$, where $b = 0$. □

Neither side condition of Theorem 5 can be dropped, on key columns or elsewhere (only `IN`-lists of two constants outside the key are left open):

Theorem 7 (Inequalities and IN-lists)**.** *Deciding* $\Sigma \models \{\tau\} : \mathrm{U}(K)$ *is coNP-complete for one-template conditions when constraints of* Σ *may have* $\notin$*-entries on columns of* K *or on other value-forced columns, and also when* τ *may have entries* $\in S$ *with* $|S| = 2$ *on columns of* K *or* $|S| = 3$ *on other value-forced columns.*

The cases on K reduce from CNF satisfiability as Theorem 6 does (Appendix A). The cases outside K reduce from 3-colourability: take `NOT NULL` columns A_u, one per vertex, and B, the goal $\{\langle\rangle\} : \mathrm{U}(B)$, and for every edge uv and colour c the constraint $\{\langle A_u{:}\notin\{c\}, A_v{:}\notin\{c\}\rangle\} : \mathrm{U}(B)$ (for $\in$: τ has $\in\{1,2,3\}$ on every A_u, and the constraint $\in\{1,2,3\} \setminus \{c\}$). A pair violating the goal satisfies the constraint for uv and c iff c is among its four values on A_u and A_v. Given a proper colouring, let both records carry on A_u the two colours other than that of u: every edge then shows all three. Conversely, two records show at most two colours per vertex; colour each vertex with one that its records miss, and no edge is monochromatic, as its endpoints together show every colour. So $\Sigma \models \{\tau\} : \mathrm{U}(B)$ iff the graph is not 3-colourable; we checked both reductions by brute force on 240 random graphs.

The general procedure: Without the side conditions the argument of Theorem 5 still applies column by column. Whether a pair satisfies Σ is a monotone function of per-column features—"a record fails this entry", "the records separate on this key column"—so on each column only states with a maximal feature set matter, and the values worth trying are the constants mentioned on the column, two fresh values and `null`. With μ_A maximal states on column A, implication is decided in time $O(|C|^2 \cdot \prod_A \mu_A \cdot |\Sigma| \cdot |R|)$; Theorems 5 and 6 are the cases $\mu_A = 1$ and $\mu_A \leq 2$. For indexes whose predicates have several templates (`OR`) the features are kept per record, and implication is coNP-complete for both semantics already for a single premise: a record escapes such a scope only by failing every template, which encodes clauses. We implemented this procedure for the full language. It agrees with exhaustive search over all pairs on 3,000 random instances with `IN`, `NOT IN`, both semantics and up to two templates per condition, and with a SAT encoding of the existence of a violating pair on 3,000 instances without $\notin$. Under the side conditions of Theorem 5 no column had two maximal states in 10,640 random instances, and Theorems 5–7 were checked against SAT on 40,000 random instances and the reductions on 700 random 3-CNFs.

Axiomatisation: Implication also has a finite axiomatisation: seven rules, sound and complete for the full language under both `null` semantics and any mix of them (Appendix C). Its completeness proof rests on a *cell lemma*: fixing the constants of an instance, every record has a *profile* (per column a constant, `null`, or "another value"), and Σ implies a goal iff, for every two profiles in the goal's scope, the pair of records of these profiles that agrees on the fewest columns violates some member of Σ. The theory assumes infinite domains: for finite types such as `boolean` every decision is sound, but an implication that holds only because a type is finite can be missed.

Refusals have witnesses: A cell that no premise covers yields a two-record counterexample: a table that satisfies every unique index, `NOT NULL` and `CHECK` constraint of the relation, yet has two rows in the query's scope that agree on the key. Our prover's search ends at such a pair when it refuses a rewrite; it does not yet report it, but a planner could show it as the reason. On the real catalogs of Section VII-B, where the prover decides implication exactly, every rewrite it refuses has such a witness; for 167 `DISTINCT`s that ignoring the `null` semantics would remove, the data itself holds one.

What the theory says to the planner: In the schemas of Section II-A, all 168 predicates are one-template conditions, and the hard cases do not arise. Taking each of the 1,480 unique indexes that are not primary keys and have plain-column keys in turn as the goal, with the other unique indexes of its table as Σ, under both semantics, 2,953 of the 2,960 implications needed one canonical pair per pair of templates, as in the case of Theorem 5; this is an observation, not membership in that case, as it includes all 34 on the six tables with <> predicates. The other 7 (all $\mathrm{U}^{=}()$) had a single column with more than one maximal state. A decision took 0.04 ms in the median and at most 2.9 ms in Python. Under these side conditions, group-shaped rewrites are hard only through nullable grouping columns left open by the query; for a `DISTINCT` over `NOT NULL` columns, or a `GROUP BY` whose block filters its columns with `IS NOT NULL` or equalities, $b = 0$. And the 84 indexes on plain columns that only exclude `null` keys are recognised as unconditional keys, which PostgreSQL currently ignores for being partial.

V. A Uniqueness Prover for the Planner

Interface: PostgreSQL's planner asks for uniqueness in one function, `relation_has_unique_index_for(root, rel, clauses, ...)`, plus its cheap pre-check `rel_supports_distinctness` and the `GROUP BY` pass `remove_useless_groupby_columns`. Each asks whether a base relation `rel` is unique on the columns equated by `clauses`. Our prover replaces the line that skips partial indexes by a call

$$\textsc{Unique}(\Sigma_{rel}, NN_{rel}, C_{rel}, K, mode)$$

where Σ_{rel} are all unique indexes of the relation with translatable predicates, C_{rel} is the condition of the relation's *base* restrictions (`baserestrictinfo`)—never join clauses, which removes the cause of bug #17975—and $mode$ is $\mathrm{U}()$ for the join-shaped callers and $\mathrm{U}^{=}()$ for the `GROUP BY` pass.

TABLE III
REWRITES AND THE CONSTRAINT THEY NEED ON THE RESTRICTED RELATION

Rewrite	Needs
Left-join removal (no inner column used)	$C : \mathrm{U}(K)$
`IN`/`EXISTS` semi-join to join	$C : \mathrm{U}(K)$
Inner-unique join†	$C : \mathrm{U}(K)$
Self-join elimination (not yet hooked)	$C : \mathrm{U}(K)$
`DISTINCT` removal*	$C : \mathrm{U}^{=}(K)$
`GROUP BY` or grouping-column removal*	$C : \mathrm{U}^{=}(K)$

* also over joins † only with `enable_conditional_unique_costing`

Translation: Index predicates are translated exactly or the index is ignored (in the schemas of Section II-A, only indexes with expression keys are); query restrictions are over-approximated: a range comparison such as `x > 3`, or any other conjunct the language cannot express, is dropped, which only makes the scope larger and so keeps every answer sound; reading it as $\top$ on its column would not, as `x IS DISTINCT FROM 3` admits `null`.

Decision: The prover runs the general procedure, which needs one canonical pair per pair of templates in the case of Theorem 5. Otherwise it searches the column states depth first, keeping maximal feature sets only, and answers "not proven" after 10^5 nodes, which loses proofs but never soundness. On one relation it is thus exact for the indexes it translates until the search gives up; the premise filter and template limit below, and the rules for joins, which are sufficient but not complete, lose further proofs, never soundness.

Rewrites: Table III lists the rewrites and their goals.

Implementation: We implemented the prover in PostgreSQL 18.6: one new file of 1,683 lines of C, 38 changed lines in six planner files, and two settings: `enable_conditional_unique` and `enable_conditional_unique_costing`, off by default, which lets a proof over a filtered index also mark a join `Inner Unique`: the executor then stops the inner scan after the first match and the planner costs the join as one-to-one, physical choices that can hurt before data is cached (Section VII-D). When no unique index without predicate settles a question, `relation_has_unique_index_ext` asks the prover. This one test serves left-join removal, inner-unique joins, semi-join reduction and the unique-ification of `IN` sub-queries; `rel_supports_distinctness` accepts partial unique indexes. A second `GROUP BY` pass runs after the restriction clauses are distributed, before pathkeys are computed, with a $\mathrm{U}^{=}()$ goal, and skips relations that an outer join can null (a null-extended row agrees with any row whose key is `null`). It also uses keys without predicates, whose columns the restriction may fix: `WHERE ns = 0` makes `UNIQUE (ns, title)` a key on `title`. The goal's condition comes from the relation's own restriction clauses, never from join clauses, which caused bug #17975. Soundness rests on three translation rules. An untranslatable conjunct is dropped from the goal, which only strengthens it, and makes an index unusable as a premise. Every equality belongs to the default btree operator family of the column's type, cross-type members included (`int2 = int4`, as generated for ORM enum columns), under a deterministic collation, and an index may use any operator class with the same equality (`text_pattern_ops`, say). Constants are identified by the family's equality, not by their representation. Templates are limited to 16, and an index is a premise only if its key lies in the goal's key or in the columns the goal fixes to one constant or to `null`; dropping premises loses proofs, never soundness. Which indexes can serve as premises at all (no expressions, the default equality on every key column) needs catalog lookups but not the question, so it is computed once per relation and planning run and kept in a new field of `RelOptInfo`; only the premise filter and the translation of the surviving predicates are done per question. PostgreSQL removes no `DISTINCT` over a base relation, with or without a predicate; we add the rewrite. Before `DISTINCT` paths are built, the prover is asked for a $\mathrm{U}^{=}()$ goal on the `DISTINCT` columns if the query reads a single base relation and has no `DISTINCT ON`, grouping, aggregates, window functions, set operations or set-returning functions in its target list. A proof means that the rows reaching `DISTINCT` are already distinct, and the operator is dropped. `DISTINCT` items other than plain columns are ignored: they can only make rows more distinct.

CHECK constraints: A validated, enforced `CHECK` constraint is a fact about every row, so it narrows the goal's scope as a restriction does. A row passes `CHECK` c when c is true or `null`, so the scope is narrowed by c or by "a column that c compares is `null`"; columns that c only tests with `IS [NOT] NULL` or `IS TRUE/FALSE` cannot make it `null`. MusicBrainz's `CHECK ((locale IS NULL AND primary_for_locale IS FALSE) OR locale IS NOT NULL)`, for instance, gives every primary alias a locale. `NOT VALID` and `NOT ENFORCED` constraints, which old rows may violate, are ignored.

Through joins: Uniqueness also carries through joins. Call a relation *determined* by the `DISTINCT` or `GROUP BY` columns G if its part of every joined row, one of its rows or nulls, is a function of the values of G. A relation is determined if its rows passing its restrictions are unique, nulls equal ($\mathrm{U}^{=}()$), on its columns in G that no outer join can null, or unique ($\mathrm{U}()$) on columns equated with columns of determined relations, by an equivalence class with two or more members or by an outer join clause read from a determined outer side; equality is strict, so $\mathrm{U}()$ suffices there. Inner sides of semi- and anti-joins add no columns to a joined row and need not be determined, but their columns determine nothing, and a block with a full join is left alone. A relation on the nullable side of a left join is determined only once its whole outer side is, because the `ON` clause decides whether a row is null-extended and may read any outer column. Single-member equivalence classes, which the planner builds for sort keys, equate nothing.

If every other relation is determined, `DISTINCT` is dropped, and so is a `GROUP BY` without aggregates; with aggregates, the `GROUP BY` columns of a relation that the other columns determine are dropped, as long as one remains. The pass uses keys without predicates too, which stock PostgreSQL uses for `GROUP BY` within one relation only.

Lemma 8 (Determination)**.** *Let the relations of a query block be joined by inner, left, semi- and anti-joins, and let the database satisfy the unique indexes, `NOT NULL` and validated `CHECK` constraints of every relation. If a relation S is determined by G under the rules above, then any two rows of the join that agree on G (`null` equal to `null`) agree on the part of S: the same row of S, or `null`s in both. If every relation outside the inner sides of semi- and anti-joins is determined, no two rows of the join agree on G.*

The proof is by induction on the order in which relations become determined; each rule supplies one step, and the side conditions are exactly what the step needs (Appendix B). Deriving the keys of a join from those of its inputs is classical [44], [21], [22]; what filtered keys add is that the premise of each step holds only under the relation's own restrictions, and that the step's semantics changes along the way: $\mathrm{U}^{=}()$ where a relation is entered through the grouping columns, $\mathrm{U}()$ where it is entered through an equality. The planner adds traps of its own. Our differential testing caught two errors in our first version (single-member classes read as equalities, an outer join entered from a partly known outer side), and TPC-DS caught a third (semi-join columns read as fixed values); Section VII-A reports how the final version and these mutants fare.

In a second optimiser: The prover is not tied to PostgreSQL. CockroachDB's optimiser removes joins, `DISTINCT` and grouping columns and carries keys through joins, all from the functional dependencies of each expression, and leaves partial unique indexes out of them (Section II-B). We ported the prover to Go (793 lines) on CockroachDB 26.2.9, the newest release in its public repository, whose stock optimiser behaves as 26.3 in our probes, and call it, behind a session setting, where the dependencies of a filter over a table scan are built (4 changed lines in the optimiser, 31 for the setting): a key it proves is added as strict if it holds with `null`s equal and as lax if only with `null`s distinct, the two kinds CockroachDB already distinguishes. Filters that read another table stay in the join, so the prover sees base restrictions only. One gap is CockroachDB's own: a lax key of an inner relation does not become a strict key of a join whose equality makes it non-null, so the running example keeps its `DISTINCT` unless `user_id` is `NOT NULL`.

VI. Recommending Filtered Unique Indexes

A filtered unique index that an application enforces only in code, or that nobody declared, can be recommended. A recommendation is a condition and a key. Conditions are cheap to propose from two sources: the schema (`IS [NOT] NULL` on nullable columns, boolean values, the values of status-, type- and kind-like columns) and the workload (the atoms its queries apply to the table). Keys under a condition come from data, and there a reduction does the work.

Proposition 9 (Reduction)**.** *For a condition C, the minimal keys K such that $C : \mathrm{U}(K)$ (respectively $C : \mathrm{U}^{=}(K)$) holds on a table r are the minimal unique column combinations of the records of r that match C, after each `null` is replaced by a value of its own (respectively all `null`s by one value).*

Any algorithm for unique column combinations [43], [3] therefore discovers conditional keys, and the complexity carries over: whether a key of at most k columns exists is NP-complete, and W[2]-complete in k, already for the empty condition [4], while the conditions of at most a atoms are polynomially many for fixed a. Without such a bound the problem degenerates: conditions over the constants of r can single out any set of rows, so the largest scope on which a key is unique is found by keeping one row per key value. The difficulty in practice is not search but *accidental* keys, which hold on the data but not by design. A recommender should prefer conditions that the workload applies and keys that keep holding as the data changes. Appendix D gives the theory and two miners: one fixes the condition and follows the hybrid scheme of [43], the other fixes the key and finds all valid conditions of at most two atoms in one pass over the pairs of records that agree on it.

VII. Evaluation

We ask whether the prover is sound (VII-A), which decisions it makes that systems miss (VII-B), what it costs (VII-C), what it gains (VII-D), and where the keys we declare on Stack Exchange and TPC-DS come from (VII-E). All runs used one core of an Apple M5 Max with 36 GB of memory; PostgreSQL ran with default settings unless stated.

A. Soundness

Regression and feature tests: With the setting off, all 231 regression tests of PostgreSQL 18.6 pass; only the list of settings changes. With the setting on for every test, no result changes; the pass through joins drops 11 `GROUP BY`s and `DISTINCT`s in the tests of aggregation and join removal, all correctly. With the `Inner Unique` setting on as well, one more plan changes: a test that a partial index `UNIQUE (id2) WHERE id2 IS NOT NULL` must not prove an inner join unique now gets `Inner Unique`, which is correct, because the join on `id2` admits only non-null values. Our own 35 feature tests compare plans and results with the setting off and on. They include the shape of bug #17975, a predicate that holds only through a join clause, where the join is kept, the union of two per-status scopes, where it is kept as well, the running example of Fig. 1 over inner, left, semi- and full joins, and `CHECK` constraints that `null`s pass, that are `NOT VALID` or `NOT ENFORCED`, where nothing is removed, and keys without predicates whose columns a restriction fixes.

Differential testing: We generated 5,000 random schemas: 3–6 columns over small domains; 1–4 unique indexes, mostly partial, with random predicates over =, <>, IN, NOT IN, IS [NOT] NULL, boolean columns, AND and OR; random NULLS NOT DISTINCT; and data whose keys repeat freely outside the scopes. On these we ran 200,000 random queries in seven shapes: left, inner and semi-joins, a scope through a clause of the same join, and GROUP BY over a table, an inner join and the nullable side of a left join. Each query ran with the setting off and on under random join and aggregation methods. 20,737 plans changed, among them 8,723 removed left joins, and no result changed. Ignoring the index predicates would have licensed 14,845 left-join removals, 5,269 of them wrong on the data. The test has power: a mutant that ignores index predicates changed 2,706 of 20,000 results. Proving GROUP BY with U() instead of $U^{=}()$ changed 232, and dropping both guards against outer-join nulling changed 3. Dropping either guard alone changed nothing, since the other one catches the case. Bug #17975 itself needs an index predicate that only another join's clause implies, as in `o JOIN t ON o.k = t.k JOIN u ON t.c = u.c` over `UNIQUE (k) WHERE c IS NOT NULL`: on 200,000 such queries the prover changed 6,713 plans and no result, while a mutant that, like PostgreSQL before 2023, accepts an index whose predicate the planner proved from any clause changed 442 of 20,000 results. A second run of 200,000 queries, SELECT DISTINCT over a table or an inner join with a random scope, changed 27,809 plans, removed 26,331 DISTINCTs, and changed no result; ignoring the index predicates would have removed 78,619, 23,796 of them wrongly, and proving DISTINCT with U() instead of $U^{=}()$ changed 551 of 20,000 results. After the premise cache was added, 80,000 further queries of each of the two kinds changed no result. A third run tested uniqueness through joins: 200,000 queries with DISTINCT, or GROUP BY with and without `count(*)`, over inner and left joins of two and three tables that all carry random unique indexes, ON clauses that may read two outer tables, and a semi-join a third of the time. 30,011 plans changed, 3,031 DISTINCTs and GROUP BYs were dropped and 12,829 grouping keys shortened, and no result changed. Three mutants restore the errors of our first version: reading single-member classes as equalities changed 34 of 20,000 results, reading semi-join columns as fixed values (found on TPC-DS q35) changed 11, and entering an outer join from a partly known outer side changed none of 40,000, as it needs an undetermined outer row that decides a match; feature tests with such data catch the last two. Two final runs of 200,000 queries each, all shapes so far with 0–2 random CHECK constraints per table (enforced, NOT ENFORCED, or added NOT VALID after rows that violate them), changed 44,887 and 31,088 plans and no result; reading a CHECK as if `null`s failed it changed 120 of 20,000 results, and trusting unvalidated constraints 327. All these runs had the `Inner Unique` setting on, which only adds to what the default does; under the default, 80,000 further queries of all shapes changed 14,351 plans and no result, and 80,000 more after the single-table GROUP BY pass was extended to tables without filtered indexes changed 13,862 plans and no result. The five main runs make up the 1,000,000 queries and 154,532 changed plans of the introduction; with the further runs, that of bug #17975 among them, no result changed on 1,520,000.

B. Decisions

For nine tables of four real datasets (Hockey, VOC, NCVoter with 1.02M records, UniProt) we declared up to 40 conditional keys per table, mined with the key-first miner of Section VI, the way an application would, as filtered unique indexes only: NULLS NOT DISTINCT for keys that hold with `null` equal to `null`, one index per `null` pattern on the key (Proposition 3) for certain keys [35], which hold whatever value a `null` stands for and so imply that, and no CHECKs. That gives 32–166 indexes per table; 19 UniProt indexes exceed PostgreSQL's 8 kB index-row limit and were left out everywhere. We generated 1,454 left joins and 1,454 DISTINCT queries whose predicates differ from the index predicates: a scope refined by an atom of another index, a key extended by a column, the union of two scopes, and random controls. We asked the real optimisers, reading their plans, and checked every decision on the data. Table IV shows the result. Stock PostgreSQL 18 uses no partial unique index. PostgreSQL 15.3, the last release before the fix for bug #17975, removes no join either: it computes `predOK` only after join removal. It marks 1,033 joins `Inner Unique`, one index at a time. SQLite 3.54 removes 33 joins by syntactic predicate matching, and on 14 queries it fails with an internal planner error: two partial unique indexes on the same column with disjoint predicates and an OR of both in the ON clause are enough to trigger it. The patched planner removes 1,094 joins and agrees with implication from the whole catalog on all 1,454 queries; it gains 61 joins over one index at a time. Neither stock system removes DISTINCT over a base table. The patched planner removes 171, none wrong, exactly those that implication from the whole catalog licenses, and 54 more than one index at a time. The other removable rewrites are removable on this data only: the catalog admits a table on which they are wrong, so no sound planner may apply them, and the patched planner is exact for what these catalogs imply. Ignoring the `null` semantics—using each index for DISTINCT as if it had NULLS NOT DISTINCT—licenses 1,033, of which 167 are wrong—113 on NCVoter, where nulls-distinct keys over sparse name suffixes put records without a suffix into one DISTINCT group. Combining indexes pays off on catalogs this dense; on the real workloads of Section VII-D it never mattered: a build that tries one premise index at a time, with restrictions, NOT NULL and CHECKs still combined, makes all 46 plan changes there that need a filtered key with the `Inner Unique` setting on (37 on Stack Exchange, 3 in Discourse, 6 in application code). CockroachDB has no NULLS NOT DISTINCT, so there every key becomes nulls-distinct indexes, one per `null` pattern but the all-`null` one (Proposition 3); this catalog implies fewer DISTINCTs, and more left joins, as the 19 oversized UniProt indexes fit CockroachDB. On the

TABLE IV
REWRITES LICENSED (WRONG) ON CATALOGS OF FILTERED UNIQUE INDEXES ONLY: 1,454 LEFT JOINS (1,208 REMOVABLE ON THE DATA) AND 1,454 DISTINCTS (985 REMOVABLE).

decider	left join	DISTINCT
PostgreSQL 18.6	0	0
PostgreSQL 15.3 (pre-#17975)[a]	0	0
SQLite 3.54	33 (14 errors)	0
one index at a time (model)	1,033	117
patched PostgreSQL 18.6	1,094	171
whole catalog (model)	1,094	171
one at a time, nulls ignored (model)	1,033	1,033 (167)

[a] marks 1,033 joins `Inner Unique`, none wrong.

same queries the stock optimiser removes 6 left joins, all joins that can never match, and no DISTINCT; the port removes 1,146 of the 1,148 left joins and all 83 DISTINCTs that the catalog implies, none wrong, against 1,087 and none one index at a time. The 2 it misses it refuses soundly, at the search limit or for want of a candidate key.

On open data: From the USA and Singapore archives of LakeBench [13] we took 285 tables whose first rows show a status-like column, an identifier and partly null columns; 217 of them (738–210,552 rows) have verified conditional keys, 6,917 in all, declared as 15,173 indexes, of which 119 exceed the index-row limit and are left out. On 29,408 left joins and as many DISTINCTs generated as above, the patched planner agrees with implication from the whole catalog on every query: it removes 23,692 joins and 4,962 DISTINCTs, none wrong, 1,483 and 963 more than one index at a time. Stock PostgreSQL removes none; SQLite removes 4,192 joins and fails on 1,123 queries over 50 tables with the internal error above. Ignoring the null semantics removes 22,171 DISTINCTs, 3,121 of them wrongly, on 133 of the 217 tables. On 40 of them, the CockroachDB port makes 4,535 of the 4,543 join removals and 225 of the 229 DISTINCT removals the catalog implies, none wrong, and refuses the other 12.

C. Cost

On the 97 TPC-DS queries that PostgreSQL can plan (scale factor 1), with the five current-version business keys declared as partial unique indexes, planning took 0.14 ms in the median and the prover added 0.026 ms in the median, a fifth of it, and 0.29 ms at most (medians of 15 runs), most of it in the pass through joins, which runs for every query that groups or de-duplicates. As a stress test we declared the keys mined on the dimension tables that hold at scale factors 1 and 10 (Section VII-E), all 3,996 of the 4,002 that a unique index can state, 4,001 indexes with the five, up to 1,771 indexes on one table. Planning then took 1.5 ms in the median without the prover, and the prover added 0.15 ms in the median, a tenth, and 2.8 ms at most; the 95th percentile of the ratio of planning times was 2.4. Without the premise cache of Section V the maximum was 45 ms, on queries that join item with its 948 indexes and plan in 2–8 ms without the prover, and without the premise filter it was 830 ms. Two TPC-DS plans changed, q46 and q68, whose GROUP BY loses ca_city of a customer address joined on its primary key; no filtered key changed a plan, for the reason given in Section VII-D.

TABLE V
WORKLOADS BY WHO WROTE THE QUERIES

workload	written by	plan	scope	changed	speed-up
code of 7 applications	developers	90	–	6 (6)	0.93
2 dbt projects (left joins)	analysts	665	12	–	–
Discourse reports	administrators	722	39	29 (2)	0.96–1.06
MusicBrainz, ListenBrainz	developers	20	0	0	–
MetaBrainz forum	users	15	0	0	–
TPC-DS	benchmark	97	0	2 (0)	–
STATS-CEB	benchmark	146	40	0	–
Data Explorer queries	users	6,578	555	118 (13)	0.37–3.30
through a filtered key				13	0.91–1.07
SkyServer SQL form	astronomers	17,738	757	26 (0)	–
Wikimedia Quarry	editors	8,736	14	178 (0)	–
soft delete, tenants	us	5	5	5 (5)	1.01–5.75
TPC-DS through views, SF 10	us	73	73	73 (73)	0.91–1.43
MusicBrainz consumers	us	10	10	7 (7)	1.15–4.94

D. Gains

Table V summarises the workloads by who wrote their queries. For each it gives the queries that plan, those that name a table with a filtered unique index together with the columns of its predicate (for SEDE, the stricter count of queries in which a table's own restriction implies the predicate), the plans that change with the prover on (in parentheses, those that change only if the filtered indexes are declared), and the speed-up of the changed queries that take over 1 ms (medians of ten rounds alternating the setting, for SEDE with the data cached; for soft delete medians of five runs after two warm-ups; through views one run after a warm-up), all under the default. The `Inner Unique` setting changes 24 more plans on Stack Exchange and one more in Discourse, all through filtered keys, and two over MusicBrainz. Every changed query returned the same result, compared in one transaction and without a trailing LIMIT, whose ties a shorter key may break differently.

Application code: Code written for an application rarely reads filtered keys by their scopes. Of the 325 code sites that read a table with one in seven applications (with the SQL ActiveRecord generates), 221 are point queries, upserts and uniqueness validations: the indexes serve integrity first. On generated data every changed query runs in under 1 ms except one Chatwoot query (0.93×). The MusicBrainz server picks a primary alias in its own code, and ListenBrainz reaches artists by identifier. Discourse's administrators join e-mail addresses on primary in 22 reports, for every one of which the prover proves the join unique, yet only 2 plans change because of it (3 with the `Inner Unique` setting): the reports de-duplicate over common table expressions, expressions or aggregates that no base-table key reaches; 27 of the 29 plan changes there come from primary keys carried through joins. dbt materialises its models, so the planner sees their joins once, and all 12 joins to current-version scopes fetch columns. TPC-DS declares current-version keys but never selects current versions, and the

146 queries of STATS-CEB [8] select a recommended scope in 40 but count over joins on non-key columns. The workloads also exposed two translation gaps, now closed: `smallint` enum columns compared with `integer` constants (GitLab) and indexes with `text_pattern_ops` (Mastodon). Users who query their own copy of MusicBrainz are no different: the 15 queries posted on the MetaBrainz forum that plan name no scope, and none changes plan.

Queries written against the data: These filter on the scopes more often. The SEDE corpus [11] holds 12,023 queries that users of the Stack Exchange Data Explorer wrote against the schema of Section VII-E; 6,578 translate from T-SQL and plan on the `stats.stackexchange.com` data with the 56 filtered unique indexes our recommender proposed (Section VII-E). A third mention a table with such an index and its predicate columns; in 555 the table's own restriction implies the predicate, most often users with a location, accepted answers, close events and duplicate links. Under the default the prover changes 118 plans: 105 with primary keys alone, through joins or in `DISTINCT` removal, and 13 because of a recommended key. With 4 GB of shared buffers, which hold the 2.8 GB of data, 15 of the 104 changed queries that take over 1 ms run more than $1.1\times$ faster (up to $3.30\times$), all through primary keys, and 1 more than $1.1\times$ slower, taking 13 ms instead of 5; the 10 timed changes through a filtered key run at 0.91–$1.07\times$. With PostgreSQL's default of 128 MB, 10 run faster and 4 slower. The `Inner Unique` setting changes 24 more plans, 142 in all and 37 through a recommended key: 28 through such a restriction, one through a restriction that the planner derives from equalities with a constant (`PostHistoryTypeId = Pt.Id = 2`), and 8 through joins on the accepted answer, whose key excludes only `null` keys: an unconditional key under U(), which PostgreSQL ignores for being partial. With the data cached, 8 of the 32 timed changes through a filtered key are then faster (up to $1.67\times$) and 3 slower (down to $0.88\times$); at 128 MB, 13 of all changed queries run faster and 15 slower. The slower ones are physical choices the cost model makes with the new knowledge, a hash join instead of a merge join over an index, or, where a `GROUP BY` goes and rows need no longer arrive in key order, a sequential scan instead of an index scan; with the data out of the cache these choices cost, with it they mostly do not. On Stack Exchange, the recommended keys pay off through join costing only; whether that is worth its risk depends on the cache, so `enable_conditional_unique_costing` leaves it to the administrator.

Astronomers who query the Sloan Digital Sky Survey through SkyServer [10] filter on scopes all the time, as its schema is built on them: `PhotoPrimary` is `PhotoObjAll` with `mode = 1` and documented to hold one row per object on the sky, `UNIQUE (thingID) WHERE mode = 1`; `SpecObj` is `SpecObjAll` with `sciencePrimary = 1`. Of the 17,738 distinct queries typed into its SQL form from January 2025 to July 2026 that plan on its schema, 10,444 read science-primary spectra, but only 1,158 de-duplicate,

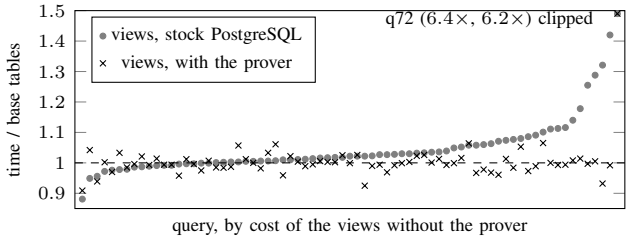


Fig. 2. TPC-DS at scale factor 10 through views that restate each history-keeping dimension with its current attributes: run time of the 73 queries that read one, relative to the same query on the base tables.

group or left-join, and the prover changes 26 plans, all through primary keys carried through joins. No query needs the documented key. 376 left-join `SpecObj` on `bestObjID` as if it were a key of the science-primary scope; in the data it is not, as 69 objects have two such spectra. A planner must prove uniqueness from declared constraints, not from what a schema is meant to say. Wikimedia's Quarry users [9] behave alike: of their queries, 8,736 plan on MediaWiki's schema and 178 change, mostly a `GROUP BY` or `DISTINCT` on `page_title` within one namespace, where the key `(page_namespace, page_title)` is a key on the title under the query's restriction, which PostgreSQL does not see. The 26 keys we mined on Simple English Wikipedia that also hold on Afrikaans Wikipedia change none; 14 queries select their scopes.

Over MusicBrainz, the queries a consumer of its data would write gain most where a partial index is a key outright ($4.94\times$ for distinct names and comments) and where an unused join to the primary alias is removed ($3.58\times$); one needs the `CHECK` constraint that gives every primary alias a locale. On soft-deleted and tenant-scoped data (users 1M rows, orders 5M), removing a left join gains 1.64 and $4.46\times$, and removing a `DISTINCT` $5.75\times$; an earlier run with the same plans gave 3.65, 5.61 and $3.87\times$, so these factors vary by up to $2\times$ between runs, and the times include fetching up to 5M rows, which no rewrite changes. The `Inner Unique` setting changes two more plans over MusicBrainz, both to 1.2–$1.8\times$ faster, and none of the others.

Views over versioned data: BI layers restate history with current attributes, so we offered each history-keeping dimension of TPC-DS as a view of the same name, `SELECT h.*, c.i_brand AS cur_i_brand, ... FROM item h LEFT JOIN item c ON c.i_item_id = h.i_item_id AND c.i_rec_end_date IS NULL`, ran the unchanged queries through it with the five current-version keys declared, and compared with the base tables (Fig. 2). Stock PostgreSQL keeps every join the views add, 141 in the 73 queries that read such a dimension at scale factor 10, and the prover removes all of them, with identical results. Without q72, which the view's `FROM` list keeps from being flattened whether or not the join goes (it is fast again with `from_collapse_limit` at 20), the views cost 7% with stock PostgreSQL (836 against 783 s) and nothing with

the prover (778 s); at scale factor 1 they cost 2% (17% with q6, whose time the views raise from 29 to 48 s). The joins go whatever their size, but what they cost depends on the dimension, and TPC-DS dimensions are small next to its facts (`item` has 102,000 rows at scale factor 10, `store_sales` 28.8 million).

E. Recommendation

Would a recommender find them?: We hid the declared indexes and generated candidate conditions from two sources. Schema roles propose `IS [NOT] NULL` on nullable columns, boolean values, and constants of status-, type- and kind-like columns. The workload contributes the atoms its queries apply, constants included. Of the 168 declared conditions on plain columns, half (84) only exclude `null` keys and follow from the key itself. Of the other 84, schema roles recover 78 (at the level of column and atom kind; the constants come from the data), the workload 68 with their constants, and both together 79. The misses are non-empty-string conditions (`<> ''`) and two predicates our parser does not read. On the indexed tables schema roles propose 2,776 candidate atoms against 524 from the workload, so the workload is the sharper source and the schema the broader one. Choosing keys under these conditions needs data, which these applications do not publish; we leave it to deployments with real instances.

Stability under updates: We mined the 15 dimension tables of TPC-DS at scale factor 1 (11,988 keys of at most three columns under conditions of at most two atoms, 10,645 of them conditional) and kept the 4,002 that also hold at scale factor 10. DuckDB's generator produces no refresh sets, so we emulated the dimension maintenance of the specification on scale factor 1: three rounds. In each round, 5% of the current versions of the history-keeping dimensions get a successor version, 5% of the rows of the other maintained dimensions are updated in place, and 2.5% new entities are added; new attribute values are drawn from each column's existing values. After three rounds, 79% of the 3,126 conditional keys on maintained tables that held at both scale factors still hold, against 58% of those that held at scale factor 1 only. Unconditional mined keys fare worse (45% and 26%): they are mostly accidents of small tables. The keys on the seven static dimensions all hold. Holding at a second scale factor is therefore a useful but weak significance filter. Of the keys it keeps, 79% survive, against 64% of all conditional keys, and it keeps only 36% of the survivors. Because the emulation samples columns independently, it breaks correlations that the real generator keeps, so these rates are lower bounds.

A public dataset: On the Stack Exchange data of `stats.stackexchange.com` (April 2024; 5.6M rows in 8 tables), conditions from schema roles (values of type, status and flag columns, null tests, and conjunctions of one of each: 457 conditions) and keys of at most three columns from HPIValid on each scope, under both semantics, gave 107 recommendations in 147 s. Under $\mathrm{U}()$ a column that is `null` throughout a scope is a key vacuously; requiring 50 rows with the key non-null removed 46 such candidates. Mined on the data before 2020, 97 of the 103 that carry dates still hold in 2024. By our reading, 81 recommendations state invariants of the platform: a revision has one history row per kind of change; a post has one initial body, title and tag set and at most one accepted-by-asker vote; an answer is accepted by at most one question (a $\mathrm{U}^{=}()$ key among non-null values); two posts are linked once per link type; a user earns a tag badge once per class. The 26 others follow correlations, such as licence versions that follow time, and the hold-out rejects only 2 of them, together with 4 invariants that turned out false: a post can become community wiki, or a hot network question, twice. All Stack Exchange sites share the schema, so we checked the 101 recommendations that hold in 2024 on three other sites (dba, academia, travel; 0.5M posts). 86 of the 99 that can be tested hold on all of them. The 13 that fail include 7 of the 24 correlations and 6 we had read as invariants: a post can be protected, or tweeted, twice, and on some sites receive several spam votes. Other sites are thus a sharper filter than later data, but the 17 correlations that pass follow the platform's history, which all sites share; choosing among recommendations stays a decision for whoever owns the schema.

Keys from data: By Proposition 9 an off-the-shelf algorithm finds the keys under a fixed condition. On seven real tables with 21 conditions each under both semantics (288 non-trivial scopes), HyUCC and HPIValid of Desbordante [12] returned the same minimal keys as our miner on every encoded scope where the miner finished within 20 s; HPIValid took 0.1–61 ms per table in the median (Appendix D).

VIII. Related Work

Uniqueness in optimisation: Semantic query optimisation rewrites queries with integrity constraints [37], [16]; filtered unique indexes are such constraints, and the question is which rewrites they license. PostgreSQL answered it unsoundly and then not at all (bug #17975); the distinction between base restrictions and join clauses that we use for uniqueness was proposed by Rowley and is under discussion for estimates [15]. Paulley and Larson [44] derive uniqueness of query results from keys; Simmen et al. [47] and Neumann and Moerkotte [40] propagate functional dependencies for order optimisation; Kossmann et al. [38] survey dependency-based optimisation. Godfrey et al. [32] exploit constraint-like characterisations of data. None of these treats keys that hold on part of a table. Deriving the keys of a join from those of its inputs goes back to Darwen [21] and Paulley [22]; Section V adds what filtered keys need, a premise per relation that holds under its own restrictions and a semantics ($\mathrm{U}()$ or $\mathrm{U}^{=}()$) per step. Lazy, workload-driven discovery of dependencies for the optimiser [23] fits the recommender of Section VI.

Partial indexes: Stonebraker proposed partial indexes [50] and Seshadri and Swami generalised them [49]; work since then concerns their selection and use as access paths. Using a query restriction to choose a partial index is the implication test of predicate subsumption [39], [31]; using a partial *unique* index for uniqueness is an implication between constraints, which is what this paper decides.

Declared but unenforced keys: DB2 eliminates joins using declared constraints [17], and Snowflake, Databricks and Oracle remove joins (Databricks also `DISTINCT`) using primary and unique keys marked `RELY` [18], [19], [20], which the system trusts but does not check. Those keys are unconditional and may be false; filtered unique indexes are enforced but hold on part of a table, so the question is not whether to trust them but what they imply.

Testing optimisers: Our differential testing follows the logic-bug oracles of Rigger and Su [24], [25], [26], which compare a query against a semantically equivalent one; we compare the planner with the rewrites off and on, and use mutants to show that the test detects the errors the safeguards prevent.

Keys with nulls and conditions: Possible and certain keys [35], [36] and key sets [51], [33], [34] fix `null` semantics of keys; $\mathrm{U}(K)$ is the possible key, and a certain key implies $\mathrm{U}^{=}(K)$ but is not expressible; embedded uniqueness constraints [54] and contextual keys [53] scope keys by completeness or by context. CFDs [5], [28], and extended CFDs with disjunction and inequality [6], bind FDs to patterns over complete relations; over such relations and sets, a conditional key with a constant condition is a CFD, and what our model adds is `null` in conditions and keys, which filtered indexes mostly state. The hardness of Theorems 6 and 7 is the finite-domain effect of CFD implication, coNP-complete with finite domains and polynomial without [28]: `null` against a value, and an `IN`-list, act as a domain of two or three values. Discovery of unique column combinations [43], [3], [1] and of CFDs [30] informs the miner.

IX. Conclusion

A planner can trust a filtered unique index when it proves uniqueness from the table's own restrictions, all its unique indexes and its `CHECK` constraints, under the `null` semantics of the rewrite. Join-shaped rewrites are then decided in polynomial time for the predicates of almost all real decisions, group-shaped rewrites are, under the same conditions, hard only through nullable columns that the query leaves open, and every implication that fails has a two-record counterexample. A prover built on this fits into PostgreSQL's planner through 38 changed lines and into CockroachDB's through 4, costs tens of microseconds in the median, licensed exactly what real catalogs imply, and changed no result on 1,000,000 random queries, while each of the errors we made on the way was caught. Turned on, it rarely loses, and what it gains depends on who writes the queries: little in application code, transformation code and reports, which use filtered keys for integrity; up to $3.3\times$ in users' queries over Stack Exchange, mostly through keys carried through joins, as filtered keys gain there only through the `Inner Unique` setting that can hurt before data is cached and stays off by default; every join that views over versioned data add; and, on queries we wrote, up to $4.9\times$ where a partial index turns out to be a key outright. Range predicates and expression keys are not translated, derived tables are not reasoned about, and self-join elimination is not yet hooked, as it needs the clauses that established uniqueness.

Appendix A
Proofs for Reasoning

Proof of Theorem 7, cases on K. Both by reduction from CNF satisfiability with `NOT NULL` columns $A_1, \ldots, A_n, B$ and the goal $\{\tau\} : \mathrm{U}(A_1 \cdots A_n B)$, so a violating pair shares a value v_i on each A_i ("x_i is true" iff $v_i = 1$). For $\notin$: τ has $\square$ everywhere, and clause constraints $\{\rho\} : \mathrm{U}(B)$ have $\rho(A_i) = \notin\{1\}$ for a positive and $\notin\{0\}$ for a negative literal; the entry fails iff the literal is true, and a value other than 0, 1 makes no literal true. For $\in$: $\tau(A_i) = \in\{0,1\}$, and $\rho(A_i) = \in\{0\}$ for a positive and $\in\{1\}$ for a negative literal. $\square$

Proposition 10. *Deciding $\Sigma \models \{\langle\rangle\} : \mathrm{U}(B)$, and $\Sigma \models \{\langle\rangle\} : \mathrm{U}^{=}(B)$, is coNP-complete already when Σ consists of a single constraint whose condition is built from `IS NULL`, `IS NOT NULL`, `AND` and `OR`.*

Proof. Membership by Lemma 2. Hardness from CNF satisfiability with clauses $c_1, \ldots, c_m$ over $x_1, \ldots, x_n$: take nullable columns $A_1, \ldots, A_n$ and $B \in NN$, read "A_i is not `null`" as "x_i is true", and let $\Sigma = \{C : \mathrm{U}(B)\}$, where C has one template per clause c_j, with $\bot$ on A_i if x_i occurs positively in c_j and $\top$ if negatively: a record matches it iff its assignment falsifies c_j. A pair violating the goal agrees on B and satisfies Σ iff one of its records lies outside the scope of C, that is, satisfies the CNF. Hence $\Sigma \models \varphi$ iff the CNF is unsatisfiable. As $B \in NN$, the same holds for $\mathrm{U}^{=}(B)$. $\square$

Separations: Let $\sigma = \{\langle d\colon \bot\rangle\} : \mathrm{U}(k)$ over columns k, d, and suppose a set Σ of embedded uniqueness constraints $E : U$ (records total on E are unique on $U \subseteq E$) were equivalent to it. The table A with two records $(k\colon 1, d\colon \texttt{null})$ violates σ, so it violates some $E : U$ in Σ, with $d \notin E$ as both records are `null` on d. The table B with $(k\colon 1, d\colon \texttt{null})$ and $(k\colon 1, d\colon 1)$ satisfies σ but violates $E : U$, as $E \subseteq \{k\}$. For $\sigma = \{\langle s\colon \in\{c\}\rangle\} : \mathrm{U}(k)$ take A with two records $(k\colon 1, s\colon c)$ and B with two records $(k\colon 1, s\colon c')$: both are total and duplicate, so every embedded constraint that A violates B violates too, while B satisfies σ. For CFDs, two identical records satisfy every CFD and violate every conditional `UNIQUE` whose scope contains them; a single record with $A = a$ and $B \neq b$ violates the CFD $(A = a \to B = b)$ and satisfies every conditional `UNIQUE`.

Appendix B
Proof of the Determination Lemma

Fix a database that satisfies the constraints, and two rows u, v of the join of the block that agree on G, `null` equal to `null`. A row of the join consists of one part per relation outside the inner sides of semi- and anti-joins: a row of the relation, at a position of its own, or `null`s if an outer join null-extended it. We show by induction on the order in which

relations become determined that u and v have the same part of each determined relation S.

Entered through G. S is unique with `null` equal to `null` on its columns K in G among its rows that pass its restrictions and `CHECK` constraints, and no outer join can null these columns. So the parts of S in u and v are rows of S, not `null`s; both pass the restrictions, which are applied below every join; they agree on K with `null` equal to `null`, because u and v agree on G; hence they are the same row.

Entered through an equivalence class. A column s of S is in an equivalence class with two or more members, one of them a column x of a determined relation R or a column of G that no outer join nulls. The class's equalities are strict and applied wherever both columns are available, so in every row that contains a row of S, its s is the value of x and not `null`. By the induction hypothesis (or because u and v agree on G), x is the same in u and v. If both contain a row of S, the two rows pass S's restrictions and agree on all columns by which S was entered, with values, so $\mathrm{U}()$ uniqueness of S on them makes them one row. It remains that both contain a row of S or neither does. If no outer join can null S, both do. Otherwise S lies on the nullable side of left joins, and the rule requires each of their outer sides to be determined: whether the nullable side of such a join is matched or null-extended is a function of the outer row, which is the same in u and v, and of the database, which is fixed.

Entered through an outer join clause. The clause equates a column s of S, on the nullable side, with a column x of a determined relation on the outer side, and the whole outer side is determined. Given the outer row, which u and v share, the rows of S that the clause admits pass S's restrictions and have s equal to the value of x; by $\mathrm{U}()$ uniqueness at most one does. So both u and v contain it, or both are null-extended, as whether a match exists depends on the outer row and the fixed database only.

Inner sides of semi- and anti-joins contribute no part and no row: a row of the join is determined by its parts outside them. If every other relation is determined, u and v have the same part of every relation, hence are the same combination of rows at the same positions, that is, the same row of the join; so no two rows agree on G. For `GROUP BY` with aggregates, a relation S determined by the remaining grouping columns G' has the same part in all rows of a group of G', so its grouping columns do not split groups; G' is kept non-empty, since grouping by nothing yields a row even for an empty input. □

APPENDIX C
THE AXIOMATISATION

Fix the set Γ of constants of an instance. The *profile* of a record has, per column, `null`, a constant of Γ, or $*$ for a value outside Γ; it is the scope of exactly one template τ_p, with $\in\{c\}$, $\bot$ and $\notin\Gamma$ as entries. For a goal with key part κ (that is, $\mathrm{U}(K)$ or $\mathrm{U}^{=}(K)$) and two profiles p, q, let $c_\kappa(p,q)$ be the *canonical pair*: records of profiles p and q that agree on K and carry different fresh values wherever both have $*$ outside K; the *cell* (p,q) is vacuous if no pair of these profiles agrees on K. Writing $\top$ for $\notin\emptyset$, D for a condition, $\tau[A{\mapsto}e]$ for τ with entry e on A, and dropping templates with $\bot$ on a `NOT NULL` column:

(T) $\Rightarrow \emptyset : \kappa$
(S) $(D \cup \{\tau[A{\mapsto}\top], \tau[A{\mapsto}\bot]\}) : \kappa$
$\Rightarrow (D \cup \{\tau[A{\mapsto}\square]\}) : \kappa$
(V) $(D \cup \{\tau[A{\mapsto}\notin S\cup\{c\}], \tau[A{\mapsto}\in\{c\}]\}) : \kappa$
$\Rightarrow (D \cup \{\tau[A{\mapsto}\notin S]\}) : \kappa$
(I) $(D \cup \{\tau[A{\mapsto}\in\{c\}] : c \in S\}) : \kappa$
$\Rightarrow (D \cup \{\tau[A{\mapsto}\in S]\}) : \kappa$
(P) $(D \cup \{\tau\}) : \kappa,\ (D \cup \{\tau'\}) : \kappa,\ \{\tau, \tau'\} : \kappa$
$\Rightarrow (D \cup \{\tau, \tau'\}) : \kappa$
(C1) $\sigma \Rightarrow \{\tau_p\} : \kappa$ if $c_\kappa(p,p)$ violates σ
(C2) $\{\tau_p\} : \kappa,\ \{\tau_q\} : \kappa,\ \sigma \Rightarrow \{\tau_p, \tau_q\} : \kappa$
if $c_\kappa(p,q)$ violates σ

In (S) the second template is omitted on a `NOT NULL` column, in (V) $c \notin S$, and in (C1), (C2) σ is any constraint with constants in Γ, omitted when the cell is vacuous.

Theorem 11 (Axiomatisation)**.** *The rules (T), (S), (V), (I), (P), (C1) and (C2) form a sound and complete* 3*-ary inference system for the implication of conditional* `UNIQUE` *constraints with conditions built from* `=, <>, IN, NOT IN, IS [NOT] NULL, AND` *and* `OR`*, under both* `null` *semantics and any mix of them.*

The proofs below use the following notation. Values come from an infinite domain $\mathbb{V}$. We write $\top$ for $\notin\emptyset$, drop templates with $\bot$ on a column of *NN* (their scope is empty), and write $\mathrm{sc}(C)$ for the records matching a condition C and $\mathrm{consts}(\cdot)$ for the constants occurring in entries. For a key part κ with key K, records t, t' *agree under* κ, $t \approx_\kappa t'$, if for all $A \in K$: $t[A] = t'[A] \neq$ `null` (for $\mathrm{U}(K)$), respectively $t[A] = t'[A]$ with `null` equal to `null` (for $\mathrm{U}^{=}(K)$). A pair (t, t') *violates* $C : \kappa$ if $t, t' \in \mathrm{sc}(C)$ and $t \approx_\kappa t'$; this is symmetric in t, t'.

A. Profiles and cells

Fix a finite $\Gamma \subseteq \mathbb{V}$. A Γ*-profile* p assigns to each column an element of $\Gamma \cup \{*, \bot\}$, and not $\bot$ to a column of *NN*. The profile $\mathrm{prof}(t)$ of a record has $t[A]$ where $t[A] \in \Gamma$, $\bot$ where $t[A] =$ `null`, and $*$ otherwise; the template τ_p has $\in\{c\}$ where p has c, $\bot$ where p has $\bot$, and $\notin\Gamma$ where p has $*$. A profile p *matches* τ if every record of profile p does, and $\mathrm{Prof}(C)$ is the set of profiles matching a template of C.

Lemma 12 (Profiles)**.** *(a)* $\mathrm{sc}(\tau_p) = \{t : \mathrm{prof}(t) = p\}$*. (b) If* $\mathrm{consts}(\tau) \subseteq \Gamma$*, whether* t *matches* τ *depends only on* $\mathrm{prof}(t)$*; so* $\mathrm{sc}(C) = \biguplus_{p \in \mathrm{Prof}(C)} \mathrm{sc}(\tau_p)$ *when* $\mathrm{consts}(C) \subseteq \Gamma$.

Proof. (a) holds by definition. (b) On a column A, $t[A]$ matches $\square$ always, $\bot$ iff $\mathrm{prof}(t)(A) = \bot$, $\in S$ with $S \subseteq \Gamma$ iff $\mathrm{prof}(t)(A) \in S$, and $\notin S$ with $S \subseteq \Gamma$ iff $\mathrm{prof}(t)(A) \notin S \cup \{\bot\}$, as a value outside Γ lies outside S. Profile scopes are disjoint by (a). □

The cell (p,q) is *vacuous for* κ if some $A \in K$ has $p(A) \neq q(A)$, or $p(A) = q(A) = \bot$ and $\kappa = \mathrm{U}(K)$. Otherwise the *canonical pair* $c_\kappa(p,q) = (t_1, t_2)$ has, on $A \in K$, $t_1[A] = t_2[A]$ equal to $p(A)$ if $p(A) \in \Gamma$, to `null` if $p(A) = \bot$, and

to a value $v_A \notin \Gamma$ if $p(A) = *$; on $A \notin K$, the constant or `null` that each profile prescribes, and where it has $*$ a value $u^i_A \notin \Gamma$; all of v_A, u^1_A, u^2_A are distinct. Then $\mathrm{prof}(t_1) = p$, $\mathrm{prof}(t_2) = q$ and $t_1 \approx_\kappa t_2$, and a cell is vacuous iff no records of profiles p, q agree under κ.

Lemma 13 (Weakest pair)**.** *Let (p, q) be non-vacuous for κ and $\mathrm{consts}(\sigma) \subseteq \Gamma$. If $c_\kappa(p, q)$ violates σ, so does every pair (t, t') with $\mathrm{prof}(t) = p$, $\mathrm{prof}(t') = q$ and $t \approx_\kappa t'$.*

Proof. Let $\sigma = D : \lambda$ with key L and $c_\kappa(p, q) = (t_1, t_2)$. By Lemma 12(b), $t, t' \in \mathrm{sc}(D)$ as t_1, t_2 are. Let $A \in L$, where t_1, t_2 agree under λ. If $A \in K$: for $p(A) = q(A) = \perp$, κ is $\mathrm{U}^=(K)$, $t_1[A], t_2[A]$ are `null`, so λ is $\mathrm{U}^=(L)$ and $t[A] = t'[A] = \mathtt{null}$ agree; for a constant, $t[A] = t'[A]$ is that constant; for $*$, $t[A], t'[A]$ are values made equal by $t \approx_\kappa t'$. If $A \notin K$: were $p(A) = *$ or $q(A) = *$, $t_1[A], t_2[A]$ would be different and not both `null`, contradicting their agreement; so $t[A] = t_1[A]$ and $t'[A] = t_2[A]$ are prescribed by the profiles, and agree. □

Lemma 14 (Cells)**.** *Let $\varphi = C_\varphi : \kappa$ and $\Gamma \supseteq \mathrm{consts}(\Sigma \cup \{\varphi\})$. Then $\Sigma \models \varphi$ iff for all $p, q \in \mathrm{Prof}(C_\varphi)$ with (p, q) non-vacuous, some $\sigma \in \Sigma$ is violated by $c_\kappa(p, q)$.*

Proof. ($\Leftarrow$) If r satisfies Σ and (t, t') in r violates φ, then $p = \mathrm{prof}(t), q = \mathrm{prof}(t') \in \mathrm{Prof}(C_\varphi)$, (p, q) is non-vacuous, and the σ violated by $c_\kappa(p, q)$ is violated by (t, t') (Lemma 13). ($\Rightarrow$) If no $\sigma \in \Sigma$ is violated by $c_\kappa(p, q) = (t_1, t_2)$ for a non-vacuous cell, the table $\{t_1, t_2\}$ respects *NN*, satisfies Σ (its pairs are (t_1, t_2) and (t_2, t_1)) and violates φ. □

B. Soundness and completeness

Proof of soundness. (T): the empty condition has empty scope. (S), (V), (I): premise and conclusion have the same key part and scope, since on A the entries of the premise partition the records matching the entry of the conclusion ($\top$ and $\perp$ for $\Box$, only $\top$ on a `NOT NULL` column; $\notin S \cup \{c\}$ and $\in\{c\}$ for $\notin S$; the $\in\{c\}$ for $\in S$). (P): let (t, t') violate the conclusion and $X = \mathrm{sc}(D)$. If both lie in $X \cup \mathrm{sc}(\tau)$ or both in $X \cup \mathrm{sc}(\tau')$, a premise is violated; otherwise one lies in $\mathrm{sc}(\tau') \setminus X$ and the other in $\mathrm{sc}(\tau) \setminus X$, violating $\{\tau, \tau'\} : \kappa$. (C1): a violating pair of $\{\tau_p\} : \kappa$ has both profiles p (Lemma 12(a)) and agrees, so (p, p) is non-vacuous and the pair violates σ (Lemma 13). (C2): a violating pair with equal profiles violates a premise $\{\tau_p\} : \kappa$ or $\{\tau_q\} : \kappa$; otherwise, up to symmetry, its profiles are p, q and it violates σ. □

For a template τ with $\mathrm{consts}(\tau) \subseteq \Gamma$ let $T(\tau)$ be the set of τ_p with p matching τ, and $\mu(\tau) = \sum_A \mu(\tau(A))$ with $\mu(\Box) = |\Gamma| + 2$, $\mu(\notin S) = |\Gamma \setminus S|$, $\mu(\in S) = |S| - 1$, $\mu(\perp) = 0$; $\mu(\tau) = 0$ iff τ is a profile template.

Lemma 15 (Merging)**.** *From $(D \cup T(\tau)) : \kappa$, rules (S), (V), (I) derive $(D \cup \{\tau\}) : \kappa$.*

Proof. Induction on $\mu(\tau)$; for $\mu(\tau) = 0$, $T(\tau) = \{\tau\}$. Otherwise choose A with $\mu(\tau(A)) > 0$ and the premise templates $\tau_1, \ldots, \tau_k$ of the rule whose conclusion template is τ: $\tau[A{\mapsto}\top]$ and, off *NN*, $\tau[A{\mapsto}\perp]$ if $\tau(A) = \Box$; $\tau[A{\mapsto}\notin S \cup \{c\}]$, $\tau[A{\mapsto}\in\{c\}]$ for some $c \in \Gamma \setminus S$ if $\tau(A) = \notin S$; $\tau[A{\mapsto}\in\{c\}]$, $c \in S$, if $\tau(A) = \in S$ with $|S| \geq 2$. Then $\mu(\tau_i) < \mu(\tau)$ and $T(\tau) = T(\tau_1) \uplus \cdots \uplus T(\tau_k)$. With $E_i = D \cup \{\tau_1, \ldots, \tau_i\} \cup T(\tau_{i+1}) \cup \cdots \cup T(\tau_k)$, the induction hypothesis derives $E_i : \kappa$ from $E_{i-1} : \kappa$, and the rule turns $E_k : \kappa$ into $(D \cup \{\tau\}) : \kappa$. □

Proof of completeness. Let $\Sigma \models \varphi = C_\varphi : \kappa$, $\Gamma = \mathrm{consts}(\Sigma \cup \{\varphi\})$; if $C_\varphi = \emptyset$ use (T). Let $P = \mathrm{Prof}(C_\varphi)$ and $T_P = \{\tau_p : p \in P\} = \bigcup_{\theta \in C_\varphi} T(\theta)$. (1) For $p \in P$, (C1) derives $\{\tau_p\} : \kappa$, with the premise that Lemma 14 provides if (p, p) is non-vacuous. (2) For $p \neq q$ in P, (C2) derives $\{\tau_p, \tau_q\} : \kappa$ likewise. (3) For $Q \subseteq T_P$ with $|Q| \geq 3$, by induction on $|Q|$, (P) with distinct $\tau, \tau' \in Q$ and $D = Q \setminus \{\tau, \tau'\}$ derives $Q : \kappa$; so $T_P : \kappa$ is derived. (4) With $C_\varphi = \{\theta_1, \ldots, \theta_m\}$ and $F_j = \{\theta_1, \ldots, \theta_j\} \cup T(\theta_{j+1}) \cup \cdots \cup T(\theta_m)$, Lemma 15 derives $F_j : \kappa$ from $F_{j-1} : \kappa$; $F_0 = T_P$ and $F_m = C_\varphi$. □

Every derived constraint has the goal's key part and members of Σ occur only as the σ of (C1) and (C2), so the rules are complete for each semantics alone. Derivations can be doubly exponentially long ($|P|$ is exponential and step (3) uses all subsets), consistent with coNP-hardness; the decision procedures of Section IV do not search for derivations.

C. A derivation and a counterexample

Example 16 (A derived key)**.** A table has the filtered unique indexes

```
UNIQUE (s) WHERE s = 'a', UNIQUE (s) WHERE s
    <> 'a', UNIQUE ((1)) WHERE s IS NULL,
```

that is $\sigma_1 = \{\langle s{:}\in\{a\}\rangle\} : \mathrm{U}(s)$, $\sigma_2 = \{\langle s{:}\notin\{a\}\rangle\} : \mathrm{U}(s)$ and $\sigma_3 = \{\langle s{:}\perp\rangle\} : \mathrm{U}(\emptyset)$ (at most one row with s `null`). Together they enforce `UNIQUE NULLS NOT DISTINCT (s)`, $\varphi = \{\langle\rangle\} : \mathrm{U}^=(s)$, so `SELECT DISTINCT s` can drop its `DISTINCT`. Here $\Gamma = \{a\}$ and the profiles are a, $*$ and $\perp$, with templates $\langle s{:}\in\{a\}\rangle$, $\langle s{:}\notin\{a\}\rangle$ and $\langle s{:}\perp\rangle$; the cells of two different profiles are vacuous, as the profiles differ on the key s. Writing conditions by their entries on s:

1	$\{\in\{a\}\} : \mathrm{U}^=(s)$	(C1), σ_1; $c(a, a)$: $s = a, a$
2	$\{\notin\{a\}\} : \mathrm{U}^=(s)$	(C1), σ_2; $c(*, *)$: $s = v, v$
3	$\{\perp\} : \mathrm{U}^=(s)$	(C1), σ_3; $c(\perp, \perp)$: $s = \mathtt{null}, \mathtt{null}$
4	$\{\in\{a\}, \notin\{a\}\} : \mathrm{U}^=(s)$	(C2), 1, 2; vacuous
5	$\{\in\{a\}, \perp\} : \mathrm{U}^=(s)$	(C2), 1, 3; vacuous
6	$\{\notin\{a\}, \perp\} : \mathrm{U}^=(s)$	(C2), 2, 3; vacuous
7	$\{\in\{a\}, \notin\{a\}, \perp\} : \mathrm{U}^=(s)$	(P), 5, 6, 4; $D = \{\perp\}$
8	$\{\top, \perp\} : \mathrm{U}^=(s)$	(V), 7; $S = \emptyset$, $c = a$
9	$\{\Box\} : \mathrm{U}^=(s)$, i.e. φ	(S), 8

Step 3 uses that two `null`s agree under $\mathrm{U}^=()$; the cell $(\perp, \perp)$ would be vacuous for a $\mathrm{U}()$ goal. Each σ_i is needed: without it, its cell has no premise.

Example 17 (A counterexample)**.** Take the two scopes of Example 4: $\sigma_1 = \{\langle d{:}\perp\rangle\} : \mathrm{U}^=(k)$, $\sigma_2 = \{\langle d{:}\top\rangle\} : \mathrm{U}(k)$, and the goal $\varphi = \{\langle x{:}1\rangle\} : \mathrm{U}(k)$, so $\Gamma = \{1\}$. Following the proof of completeness, the cells within the scope of φ are checked one by one. The cell of $p = (k{:}*, d{:}\perp, x{:}1)$ and

$q = (k{:}*, d{:}*, x{:}1)$ is non-vacuous (both have a value on k), and its canonical pair is

$$t_1 = (k{:}v,\ d{:}\texttt{null},\ x{:}1), \qquad t_2 = (k{:}v,\ d{:}u,\ x{:}1).$$

It violates neither premise: t_2 is outside the scope of σ_1 and t_1 outside that of σ_2. So (C2) has no premise for this cell, the derivation cannot be completed, and by Lemma 14 the table $\{t_1, t_2\}$ is a counterexample: it satisfies both indexes and has two rows with $x = 1$ and the same k. This is the evidence a planner obtains when it refuses to drop a join probing k among the rows with $x = 1$.

Appendix D
Discovery

For a table r, let $\mathrm{Th}(r)$ be the set of conditional `UNIQUE` constraints that r satisfies; a *cover* of $\mathrm{Th}(r)$ is a finite subset that implies all of it.

Theorem 18 (Finite covers)**.** *Let Γ be the constants occurring in r. The constraints $\{\tau_p, \tau_q\} : \kappa$ that hold on r, for Γ-profiles p, q and key parts $\kappa \in \{\mathrm{U}(K), \mathrm{U}^{=}(K)\}$ with K minimal, form a finite cover of $\mathrm{Th}(r)$; it implies also the constraints with constants that do not occur in r.*

The cover is finite because `NOT IN` names "a value that r does not contain" with one entry. But it shows that the unrestricted problem is degenerate: every value of r lies in Γ, so the profile of a row records all its values, and conditions can single out any set of rows. The cover above enumerates pairs of rows—for tables of at most five rows and three columns it has 146 members in the median and up to 4,232—and the largest scope on which a key K is unique is found trivially by keeping one row per value of K. Discovery is meaningful only over a bounded condition language. Let $\mathcal{L}_a$ be the templates with at most a entries other than $\square$, each an *atom* `A = c`, `A <> c` (c in the active domain of A), `A IS NULL` or `A IS NOT NULL`, and let $\mathrm{Disc}_a(r)$ list, for every $\tau \in \mathcal{L}_a$ and both semantics, the minimal keys K such that $\tau : \mathrm{U}(K)$, respectively $\tau : \mathrm{U}^{=}(K)$, holds on r. As a constraint implies the same constraint with a larger key, $\mathrm{Disc}_a(r)$ covers the constraints with conditions in $\mathcal{L}_a$ that hold on r.

Theorem 19 (Complexity of discovery)**.** *Let r be a table over R, and d the largest active domain of a column.*

(a) *Whether a given constraint holds on r is decidable in polynomial time.*
(b) *$|\mathcal{L}_a| \le \big(|R|\,(2d+3)\big)^a$, polynomial for fixed a.*
(c) *Given τ and k, deciding whether $\tau : \mathrm{U}(K)$ holds on r for some K with $|K| \le k$ is NP-complete and, parameterised by k, W[2]-complete, already for $\tau = \langle\rangle$; the same holds for $\mathrm{U}^{=}()$.*
(d) *For given τ and semantics, the minimal keys are the minimal transversals of a hypergraph on R with at most $|r|^2$ edges, computable in polynomial time; for $\tau = \langle\rangle$ over tables without `null`, enumerating them is equivalent under parsimonious reductions to enumerating the minimal transversals of a hypergraph [4].*

Algorithm 1 FixC(r, C, κ, k, H_0): minimal keys of at most k columns

$S \leftarrow$ the rows of r matching C; **if** $|S| < 2$ **return** $\{\emptyset\}$
replace each `null` in S by a value of its own ($\mathrm{U}()$) or by one value $\perp$ ($\mathrm{U}^{=}()$)
$\pi_A \leftarrow$ clusters of ≥ 2 rows of S equal on A, for each $A \in R$
$H \leftarrow \{D \in H_0 :$ its two rows lie in $S\} \cup \{D(t,t') : t, t'$ consecutive in a cluster$\}$
$T \leftarrow \{\emptyset\}$; **for each** $D \in \min H$: $T \leftarrow \mathrm{Add}_k(T, D)$; $V \leftarrow \emptyset$
while some $X \in T \setminus V$ **do**
  if the refinement of π by X has a cluster with rows t, t' **then**
    if $D(t,t') = \emptyset$ **return** $\emptyset$ ▷ no key exists
    $H \leftarrow H \cup \{D(t,t')\}$; $T \leftarrow \mathrm{Add}_k(T, D(t,t'))$
  else $V \leftarrow V \cup \{X\}$
  end if
end while
return T (and H, for reuse)

Algorithm 2 DiscA(r, a, k): all templates of $\mathcal{L}_a$

for $\kappa \in \{\mathrm{U}(), \mathrm{U}^{=}()\}$ and $\tau \in \mathcal{L}_a$ by increasing number of atoms **do**
  $\tau' \leftarrow \tau$ without its last atom ▷ a wider scope
  $(K_\tau, H_\tau) \leftarrow$ FixC$(r, \{\tau\}, \kappa, k, H_{\tau'})$ ▷ $H_{\langle\rangle}$ from $H_0 = \emptyset$
end for
return all $\tau : \kappa(K)$ with $K \in K_\tau$

Hence, for fixed a, $\mathrm{Disc}_a(r)$ is computable in incremental quasi-polynomial time [29], in output-polynomial time if minimal transversals can be enumerated in output-polynomial time, and for $a = 0$ only then [27]. With keys bounded by a fixed size, it takes polynomial time.

Part (c) follows from the unconditional case [4], [48], part (d) from the observation that K is a key within a scope iff it meets the set of columns on which every two rows of the scope fail to agree (proofs below). Our miner works with $a = 2$ and keys of at most three columns, so its search is polynomial; what makes discovery hard is unbounded key size, as for unconditional keys, not the conditions.

Fixing the condition: By part (d) and Proposition 9, discovery for a fixed condition is discovery of unique column combinations on the encoded rows in scope. Algorithm 1 follows the hybrid scheme of [43]: disagreement sets of neighbouring rows in the position list indexes (PLIs) give candidate keys as minimal transversals; each candidate is validated by intersecting PLIs, and a failing candidate yields a pair whose disagreement set updates the candidates by one step of Berge's algorithm [2], $\mathrm{Add}_k(T, D) = \min\big(\{X \in T : X \cap D \neq \emptyset\} \cup \{X \cup \{A\} : X \in T,\ X \cap D = \emptyset,\ A \in D,\ |X| < k\}\big)$.

Proposition 20. FixC *returns exactly the minimal keys of at most k columns, and* DiscA *returns $\mathrm{Disc}_a(r)$ restricted to such keys.*

Proof. Every disagreement set in H belongs to two rows of the scope, so every key meets it. A returned X is a validated key and a minimal transversal of H, so no proper subset of X is a key. A minimal key K with $|K| \le k$ is a transversal of H and contains a minimal one, X, with $|X| \le k$, which Add_k

keeps; X is validated, so it is a key and $X = K$. Each failed validation adds a set that the candidate misses, so H grows and the loop ends. Inherited sets in DISCA are kept only if both rows lie in the narrower scope. □

The cost is dominated by validations, $O(|S|\,|X|)$ each; on random tables both procedures agree with exhaustive search (Section D-B).

Fixing the key: Our second miner goes the other way. For a candidate key K (at most three columns) the pairs of records that agree on K are computed once by grouping; a condition makes $C : \mathrm{U}(K)$ hold iff no such pair has both records in its scope, so all valid templates of at most two atoms—a frequent constant of a low-cardinality column, a column present, a column missing—are found in one pass over the atom sets the pairs share, and single-constant templates are merged into `IN`-lists whose cross pairs are clean. Results implied by the declared indexes or by more general results are dropped by the prover of Section V, so the recommendation is irredundant.

The difficulty is not search but accidental keys. On TPC-DS's six-row `call_center` table, 342 unconditional keys hold; checking them on a second scale factor keeps 156, and on small dimension tables almost all. Ranking by scope size, identifier-like columns and simplicity of the condition helps; a recommender should, in addition, prefer keys that the workload can use.

A. Proofs

Proof of Theorem 18. Every listed constraint holds on r by definition, and there are finitely many, as there are finitely many Γ-profiles and keys. Let $\varphi = C : \kappa$ hold on r, with key K_φ, and suppose the cover Σ does not imply it. With $\Gamma' = \Gamma \cup \mathrm{consts}(\varphi)$, Lemma 14 yields Γ'-profiles $p', q' \in \mathrm{Prof}(C)$ whose cell is non-vacuous and whose canonical pair (t_1, t_2) satisfies Σ. Let p, q be the Γ-profiles of t_1, t_2 and $Z \supseteq K_\varphi$ the set of columns on which t_1, t_2 agree under κ; write κ_Z for the key part of the same semantics with key Z. The pair (t_1, t_2) violates $\{\tau_p, \tau_q\} : \kappa_Z$. If that constraint held on r, so would one with a minimal key $K \subseteq Z$, which belongs to Σ and is also violated by (t_1, t_2)—a contradiction. So r has rows t, t' at different positions with Γ-profiles in $\{p, q\}$ that agree on Z under κ. Rows of r carry only values of Γ, so these profiles contain no $*$; hence the corresponding record of the canonical pair carries only values of Γ or `null`, and its Γ'-profile, p' or q', equals its Γ-profile. Thus t, t' have Γ'-profiles in $\{p', q'\} \subseteq \mathrm{Prof}(C)$, lie in $\mathrm{sc}(C)$ by Lemma 12, and agree on $K_\varphi \subseteq Z$ under κ: (t, t') violates φ on r, a contradiction. □

The construction was checked mechanically: on 400 random tables and 2,000 random constraints, with constants inside and outside the tables, "φ holds on r" coincided with "the cover implies φ".

Proof of Theorem 19. (a) Collect the rows of the scope, drop for $\mathrm{U}(K)$ those with `null` on K, and look for two rows with equal projections on K (with `null` as a value for $\mathrm{U}^{=}(K)$) by sorting or hashing. (b) A template of $\mathcal{L}_a$ chooses at most a columns and, for each, one of at most $2d+2$ atoms; counting the choice "no atom" gives the bound. (c) Membership in NP: guess K and check it by (a). For hardness take $\tau = \langle\rangle$ and tables without `null`, on which $\mathrm{U}(K)$ and $\mathrm{U}^{=}(K)$ both say that K is a unique column combination. Deciding whether a unique column combination of size at most k exists is NP-complete and, parameterised by k, W[2]-complete [4]; finding a minimum one is NP-hard [48]. (d) For rows t, t' let $D_\kappa(t, t')$ be the set of columns on which they do not agree under κ. Then $\tau : \kappa$ holds on r iff $K \cap D_\kappa(t, t') \neq \emptyset$ for all rows t, t' at different positions in the scope of τ, i.e. iff K is a transversal of the hypergraph of these sets; the minimal keys are its minimal transversals. If some set is empty, no key exists. For $\tau = \langle\rangle$ and tables without `null` these are the minimal unique column combinations, whose enumeration is equivalent to transversal enumeration under parsimonious reductions [4]. For the consequences: $\mathrm{Disc}_a(r)$ consists of $2|\mathcal{L}_a|$ transversal problems, each built in time $O(|r|^2|R|)$; the duality test of Fredman and Khachiyan runs in quasi-polynomial time and, when it fails, returns a point from which a new minimal transversal is obtained, so minimal transversals are enumerated in incremental quasi-polynomial time [29]; an output-polynomial algorithm for transversals gives one for each problem and hence for their union, whose output is the sum of theirs; for $a = 0$ the converse holds as minimal unique column combinations can be enumerated in output-polynomial time only if transversals can [27], [4]. With $|K| \leq k$ fixed, all $O(|R|^k)$ candidate keys are checked by (a) for each template. □

Proof of Proposition 9. After the replacement, two records agree on K in the plain sense iff they agree under the chosen semantics, so $C : \kappa(K)$ holds on r iff K is unique on the records matching C. □

B. Experiments

Correctness: FIXC agreed with exhaustive search over all subsets of columns on 5,000 random tables of up to five columns and twelve rows with `null`s and duplicates, random conditions with =, <>, `IN`, `NOT IN` and `IS [NOT] NULL`, both semantics and key bounds $k \in \{1, 2, \infty\}$, and on 1,500 wider tables (four to eight columns, up to 40 rows) with up to 44 minimal keys each; DISCA with inherited disagreement sets agreed with FIXC from scratch on all 42,958 templates of $\mathcal{L}_2$ over 500 random tables.

Cost: On seven real tables (Table VI) we ran FIXC for the empty condition and 20 single-atom conditions (the most frequent constants, `IS NULL` and `IS NOT NULL` of the sparsest columns), under both semantics, with keys of at most three columns, against a level-wise baseline that validates every candidate whose subsets are not keys (single-threaded Python, same PLIs, same output on all 294 runs). FIXC needs 6–85 times fewer validations and is faster on 238 of the 294 runs, with median speed-ups of 1.6–5.7 on six tables and up to 88 on UniProt. It loses on Hockey.Teams (median 0.35):

TABLE VI
FIXC AGAINST LEVEL-WISE VALIDATION, KEYS OF ≤ 3 COLUMNS: MEDIANS OVER 42 RUNS (21 CONDITIONS, TWO SEMANTICS); H. = HOCKEY, V. = VOC.

table	rows	cols	keys	FixC ms (val.)		levels ms (val.)
H.Master	7,761	31	263	462	(359)	788 (2,268)
H.Coaches	1,812	14	4	14	(5)	55 (427)
H.Teams	1,519	27	310	704	(356)	204 (3,133)
H.Goalies	4,278	23	46	101	(53)	420 (2,005)
V.voyages	8,131	23	37	97	(37)	558 (1,094)
V.seafarers	4,468	10	3	10	(3)	15 (73)
UniProt (20k)	20,000	100	396	1,655	(410)	11,587 (24,744)

with about 300 minimal keys per condition, maintaining the transversals costs more than the validations it saves, which are cheap on 1,519 rows. Inheriting disagreement sets in DISCA (70–86 single-atom templates per table) saved at most 13% of the time against FIXC from scratch, and nothing on most tables: the seeds from PLI neighbours already find most disagreement sets, and validation dominates. Inheritance is therefore a minor optimisation.

Against UCC algorithms: On the same seven tables and 21 conditions under both semantics (288 scopes with at least two records; UniProt sampled to 20,000 of its rows), we wrote each encoded scope to a file and ran HyUCC and HPIValid of Desbordante 2.4.1 (C++, one thread) [12]. The two returned the same keys on all 288 scopes; FIXC without a size bound returned them on the 236 it finished within 20 s, and with at most three columns on the 281 it finished. HPIValid took 0.1–61 ms per table in the median, HyUCC 0.4–50,091 ms: under $\mathrm{U}^{=}()$ on UniProt, where the scopes have about 76,000 minimal keys in the median, HyUCC needed up to 21 minutes. FIXC with keys of at most three columns took 6.4 times as long as HPIValid in the median, loading included. Widening UniProt from 10 to 100 columns raised HPIValid from 0.5 to 45 ms. Our contribution to discovery is the reduction, not a new algorithm for unique column combinations.

Scaling the rows of the 30-column UniProt exports from 16,000 to 512,000 (empty condition and three `IS [NOT] NULL` conditions, both semantics; 40 scopes) kept all answers equal. HPIValid grew from 2.5 to 54 ms and HyUCC from 18 to 1,479 ms on the empty condition under $\mathrm{U}()$, while reading the file grew from 0.4 to 16 s and dominated both. FIXC with keys of at most three columns needed 0.7 s at 16,000 rows and 16 s at 256,000, and exceeded 20 s at 512,000 on the large scopes.

On 40 of the LakeBench tables of Section VII-B, cut to the miner's key columns and up to 12 condition columns (1,630 scopes of up to 163,414 rows), all four agreed on every scope and FIXC always finished; in the median per table HPIValid took 0.4 ms, HyUCC 4.3 ms and FIXC with at most three columns 77 ms.

APPENDIX E
FURTHER EXPERIMENTS

Scaling of the decision procedure: On 30-column schemas with random conditional keys of one or two atoms, our Python implementation of the canonical-pair search needs 0.15, 0.37, 2.2 and 15 ms in the median for $|\Sigma| = 10$, 100, 1,000 and 10,000 at $b = 4$, against 0.34–244 ms for a SAT encoding; for $|\Sigma| = 100$ it grows from 0.19 ms at $b = 0$ to 669 ms at $b = 16$, where SAT stays at about 2 ms. The polynomial case of Theorem 5 is the $b = 0$ column.

A rewrite layer outside the planner: A catalog-reading rewrite layer in Python reproduced all 2,588 decisions on the eight tables whose indexes PostgreSQL can build (on UniProt some index rows exceed 8 kB), and the 80 sampled rewrites it emitted returned the original results. Reading a table's constraints took 1–11 ms and deciding and rewriting a query 3.2 ms in the median in Python. The rewrites ran 1.8× (`DISTINCT`) and 3.5× (left join) faster in the median, and removing the join 10.7–32.3× on NCVoter. These runs used catalogs with all key classes of the miner, not filtered unique indexes only.

Mining the TPC-DS workload: The queries are also a source of conditions. The local predicates of each table occurrence (conjuncts of `WHERE` and inner-join `ON` clauses on that table alone, kept as SQL including ranges) give 162 distinct conditions on 22 tables in 96 of the 99 queries, extracted in 1.5 s. Checked by grouping at scale factors 1 and 10, 124 of the 181 key demands of the queries—the columns a block joins, groups or semi-joins on—hold under their condition, but 118 contain the primary key, and only 3 hold because of the condition, all on scopes of two to four rows that the query parameters select. Running FIXC under every condition on a dimension table (keys of at most two columns, 14 s in all) finds 1,983 keys that hold only under their condition, 1,382 of them also at scale factor 10, and 844 of these on scopes of fewer than 100 rows. Of the 794 under conditions of equalities and `IN`-lists, 232 hold for every value of the condition columns—d_{dom} within each month, for instance—and so are composite keys that include those columns (here $(d_{year}, d_{moy}, d_{dom})$) rather than conditional ones; the others depend on the parameter values. Mining the workload finds scopes cheaply, but a recommender needs a significance filter, and the selective, parameterised predicates of a benchmark are not where filtered unique indexes live.

ACKNOWLEDGMENT

Claude (Anthropic), through Claude Code, was used for this work. It generated text for the article, code for the PostgreSQL prover and CockroachDB port, experiment scripts, random schema and query generators (Sections V and VII); it ran the experiments, compiled their results, drew the tables and figures, checked the reductions of Section IV by brute force on random instances, and reviewed drafts. The authors directed the work and are responsible for the correctness and originality of the paper.

References

[1] Z. Abedjan, L. Golab, F. Naumann. Profiling relational data: a survey. *VLDB J.* 24(4), 2015.

[2] C. Berge. *Hypergraphs: Combinatorics of Finite Sets*. North-Holland, 1989.

[3] J. Birnick, T. Bläsius, T. Friedrich, F. Naumann, T. Papenbrock, M. Schirneck. Hitting set enumeration with partial information for unique column combination discovery. *Proc. VLDB Endow.* 13(11):2270–2283, 2020.

[4] T. Bläsius, T. Friedrich, M. Schirneck. The complexity of dependency detection and discovery in relational databases. *Theor. Comput. Sci.* 900:79–96, 2022.

[5] P. Bohannon, W. Fan, F. Geerts, X. Jia, A. Kementsietsidis. Conditional functional dependencies for data cleaning. In *ICDE*, pp. 746–755, 2007.

[6] L. Bravo, W. Fan, F. Geerts, S. Ma. Increasing the expressivity of conditional functional dependencies without extra complexity. In *ICDE*, pp. 516–525, 2008.

[7] T. Döhmen, R. Geacu, M. Hulsebos, S. Schelter. SchemaPile: a large collection of relational database schemas. *Proc. ACM Manag. Data* 2(3), 172:1–172:25, 2024.

[8] Y. Han, Z. Wu, et al. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. *PVLDB*, 15(4):752–765, 2021.

[9] H. Triedman. wikidb: queries from Wikimedia's Quarry. Hugging Face dataset `htriedman/wikidb`, 2023.

[10] A. S. Szalay, J. Gray, et al. The SDSS SkyServer: public access to the Sloan Digital Sky Server data. In *SIGMOD*, 2002.

[11] M. Hazoom, V. Malik, B. Bogin. Text-to-SQL in the wild: a naturally-occurring dataset based on Stack Exchange data. In *NLP4Prog Workshop at ACL*, 2021.

[12] G. Chernishev et al. Desbordante: from benchmarking suite to high-performance science-intensive data profiler. In *CODS-COMAD*, pp. 234–243, ACM, 2025.

[13] Y. Deng et al. LakeBench: a benchmark for discovering joinable and unionable tables in data lakes. *PVLDB*, 17(8):1925–1938, 2024.

[14] K. R. Carlsen. Postgres wont remove useless joins, when the UNIQUE index is partial. pgsql-general mailing list, January 2019.

[15] C. Yan. Restrict partial unique index estimates to base quals. pgsql-hackers mailing list, patch v1, July 2026.

[16] U. S. Chakravarthy, J. Grant, J. Minker. Logic-based approach to semantic query optimization. *ACM Trans. Database Syst.* 15(2):162–207, 1990.

[17] Q. Cheng, J. Gryz, F. Koo, T. Y. C. Leung, L. Liu, X. Qian, K. B. Schiefer. Implementation of two semantic query optimization techniques in DB2 Universal Database. In *VLDB*, pp. 687–698, 1999.

[18] Snowflake. *Join elimination*. https://docs.snowflake.com/en/user-guide/join-elimination.

[19] Databricks. *Query optimization using primary key constraints*. https://docs.databricks.com/aws/en/sql/user/queries/query-optimization-constraints.

[20] Oracle. *Database SQL Tuning Guide 19c*, chapter Query Transformations.

[21] H. Darwen. The role of functional dependence in query decomposition. In C. J. Date, H. Darwen, *Relational Database Writings 1989–1991*, Addison-Wesley, 1992.

[22] G. N. Paulley. *Exploiting Functional Dependence in Query Optimization*. PhD thesis, University of Waterloo, 2000.

[23] J. Kossmann, D. Lindner, F. Naumann, T. Papenbrock. Workload-driven, lazy discovery of data dependencies for query optimization. In *CIDR*, 2022.

[24] M. Rigger, Z. Su. Testing database engines via pivoted query synthesis. In *OSDI*, pp. 667–682, 2020.

[25] M. Rigger, Z. Su. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *ESEC/FSE*, pp. 1140–1152, 2020.

[26] M. Rigger, Z. Su. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.* 4(OOPSLA), 211:1–211:30, 2020.

[27] T. Eiter, G. Gottlob. Identifying the minimal transversals of a hypergraph and related problems. *SIAM J. Comput.* 24(6):1278–1304, 1995.

[28] W. Fan, F. Geerts, X. Jia, A. Kementsietsidis. Conditional functional dependencies for capturing data inconsistencies. *ACM Trans. Database Syst.* 33(2):6, 2008.

[29] M. L. Fredman, L. Khachiyan. On the complexity of dualization of monotone disjunctive normal forms. *J. Algorithms* 21(3):618–628, 1996.

[30] W. Fan, F. Geerts, J. Li, M. Xiong. Discovering conditional functional dependencies. *IEEE Trans. Knowl. Data Eng.* 23(5):683–698, 2011.

[31] J. Goldstein, P.-Å. Larson. Optimizing queries using materialized views: a practical, scalable solution. In *SIGMOD*, 2001.

[32] P. Godfrey, J. Gryz, C. Zuzarte. Exploiting constraint-like data characterizations in query optimization. In *SIGMOD*, 2001.

[33] S. Hartmann, U. Leck, S. Link. On Codd families of keys over incomplete relations. *Comput. J.* 54(7):1166–1180, 2011.

[34] M. Hannula, X. Li, S. Link. Controlling entity integrity with key sets. *J. Comput. Syst. Sci.* 136:195–219, 2023.

[35] H. Köhler, S. Link, X. Zhou. Possible and certain SQL keys. *Proc. VLDB Endow.* 8(11):1118–1129, 2015.

[36] H. Köhler, U. Leck, S. Link, X. Zhou. Possible and certain keys for SQL. *VLDB J.* 25(4):571–596, 2016.

[37] J. J. King. QUIST: a system for semantic query optimization in relational databases. In *VLDB*, pp. 510–517, 1981.

[38] J. Kossmann, T. Papenbrock, F. Naumann. Data dependencies for query optimization: a survey. *VLDB J.* 31(1), 2022.

[39] P.-Å. Larson, H. Z. Yang. Computing queries from derived relations. In *VLDB*, 1985.

[40] T. Neumann, G. Moerkotte. An efficient framework for order optimization. In *ICDE*, pp. 461–472, 2004.

[41] PostgreSQL Global Development Group. *PostgreSQL Documentation*: CREATE INDEX; Partial indexes. https://www.postgresql.org/docs/, accessed 2026.

[42] D. Rowley. Don't use partial unique indexes for unique proofs in the planner. PostgreSQL commit 7fcd7ef2a9, bug #17975, June 2023.

[43] T. Papenbrock, F. Naumann. A hybrid approach for efficient unique column combination discovery. In *BTW*, 2017.

[44] G. N. Paulley, P.-Å. Larson. Exploiting uniqueness in query optimization. In *ICDE*, pp. 68–79, 1994.

[45] SQLite. *Partial indexes*. https://sqlite.org/partialindex.html, accessed 2026.

[46] Microsoft. *SQL Server documentation*: CREATE INDEX; Create filtered indexes. https://learn.microsoft.com/sql/, accessed 2026.

[47] D. Simmen, E. Shekita, T. Malkemus. Fundamental techniques for order optimization. In *SIGMOD*, 1996.

[48] A. Skowron, C. Rauszer. The discernibility matrices and functions in information systems. In R. Słowiński (ed.), *Intelligent Decision Support*, Theory and Decision Library D 11, pp. 331–362. Kluwer, 1992.

[49] P. Seshadri, A. Swami. Generalized partial indexes. In *ICDE*, pp. 420–427, 1995.

[50] M. Stonebraker. The case for partial indexes. *SIGMOD Rec.* 18(4):4–11, 1989.

[51] B. Thalheim. On semantic issues connected with keys in relational databases permitting null values. *J. Inf. Process. Cybern.* 25(1/2):11–20, 1989.

[52] P. White. Optimizer limitations with filtered indexes. https://www.sql.kiwi/2013/04/optimizer-limitations-with-filtered/, 2013.

[53] Z. Wei, S. Link, J. Liu. Contextual keys. In *ER*, pp. 266–279, 2017.

[54] Z. Wei, U. Leck, S. Link. Discovery and ranking of embedded uniqueness constraints. *Proc. VLDB Endow.* 12(13):2339–2352, 2019.